UNIVERSITÀ
DI SIENA
1240

University of Siena

Department of Information Engineering and Mathematics

PathNet: A* Search for Multilayer Perceptron Training

Kevin Gagliano

Jacopo Andreucci

Supervisor:

Prof. Edmondo trentin

Academic Year 2024-2025

Contents

Chapter 1

The A* Search Algorithm

1.1 Overview and Context

The **A* (A-star) search algorithm** is a cornerstone of Artificial Intelligence and computer science, highly effective for pathfinding and optimal graph traversal. Developed in 1968 by Hart, Nilsson, and Raphael as an extension of Dijkstra's algorithm, A* is classified as an *informed search* or *best-first search* method. Unlike uninformed searches, A* strategically explores the solution space by using estimated knowledge about the goal, enabling it to efficiently find the shortest path between a start and goal node. Its classical applications include video game pathfinding, network routing, and logistical planning.

1.2 The Evaluation Function $f(n)$

The central innovation of the A* algorithm lies in its heuristic evaluation function, $f(n)$, which estimates the total cost of the path from the starting point through the current node n to the goal. This function is defined as the sum of two components: the true cost from the start, $g(n)$, and the estimated cost to the goal, $h(n)$.

$$f(n) = g(n) + h(n)$$

1.2.1 Path Cost ($g(n)$)

The function $g(n)$ represents the actual cost of the path taken from the initial starting node to the current node n. This is a known, objective value and is a direct measure of the work done so far. In traditional pathfinding, $g(n)$ might be the distance traveled;

in optimization problems like the one discussed in this report, $g(n)$ represents the product between the initial loss and the percentage of search iterations performed.

1.2.2 Heuristic Estimate ($h(n)$)

The function $h(n)$ is the heuristic estimate of the cost from the current node n to the final goal node. A heuristic is an educated guess that guides the search towards the most promising regions of the search space. The effectiveness and properties of the A* algorithm are highly dependent on the quality of this heuristic.

The heuristic $h(n)$ must satisfy the following critical property to guarantee optimality:

- **Admissibility:** A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, meaning for all nodes n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true minimum cost from n to the goal.

If an admissible heuristic is used, A* is guaranteed to find the optimal (lowest-cost) path, provided a path exists.

Consistency

The condition of consistency is a stronger requirement than admissibility. A heuristic $h(n)$ is consistent if, for every node n and every successor node n' connected by an edge with cost $c(n, n')$, the estimated cost from n to the goal is less than or equal to the cost of moving to n' plus the estimated cost from n' to the goal.

Consistency Condition: $h(n) \leq c(n, n') + h(n')$

Consistency implies admissibility, provided $h(\text{goal}) = 0$. When a consistent heuristic is used, A* is guaranteed to find the optimal path without needing to re-open nodes (re-examine a node already in the Closed Set), because the f-cost along any optimal path is monotonically non-decreasing.

- **Optimality Guarantee:** If the heuristic $h(n)$ is admissible (and edge costs are non-negative), A* is guaranteed to find the optimal (lowest-cost) path. If the heuristic is consistent, A* is also guaranteed to be optimal and more efficient, as it never needs to process a node more than once.

1.3 Algorithm Mechanics

A* operates by iteratively expanding nodes, maintaining two key data structures:

- **Open Set (or Frontier):** A priority queue containing nodes that have been generated but not yet fully explored. Nodes in the open set are prioritized based on their calculated $f(n)$ value.
- **Closed Set:** A set containing nodes that have already been fully expanded. This prevents the algorithm from revisiting and reprocessing the same node multiple times.

The core loop of the A* algorithm is as follows:

1. Initialize the Open Set with the starting node.
2. While the Open Set is not empty:
   - Select the node n from the Open Set with the lowest $f(n)$ value.
   - If n is the goal state, terminate the search.
   - Move n from the Open Set to the Closed Set.
   - Otherwise, generate all successors of n. For each successor s:
     - Calculate the cost of the path to s via n: $g_{\text{new}}(s) = g(n) + \text{cost}(n, s)$.
     - **Path Evaluation and Update:**
       * If s is in the Open Set, check if $g_{\text{new}}(s) < g(s)$. If true, update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and update its priority in the Open Set.
       * If s is in the Closed Set, check if $g_{\text{new}}(s) < g(s)$. If true, this means a cheaper path to s has been found. Update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and move s back from the Closed Set to the Open Set for re-expansion.
       * If s is not in either set, it is a newly discovered node. Set its parent to n, set $g(s) = g_{\text{new}}(s)$, calculate $f(s) = g(s) + h(s)$, and add it to the Open Set.

1.4 Properties of A*

The A* search algorithm is known for two key theoretical properties:

- **Completeness:** If a solution path exists from the start node to the goal node, A* is guaranteed to find it, provided the branching factor is finite and edge costs are non-negative.
- **Optimality:** A* is optimally efficient and guaranteed to find the lowest-cost path if the heuristic function $h(n)$ is admissible. This makes it a preferred method for problems where minimizing the cost (or, in the context of this project, minimizing the training error) is paramount.

The adaptation of A* from a simple pathfinding tool to a machine learning optimization technique, as explored in this report, requires careful consideration of the state space, the definition of the path cost $g(n)$, and the design of an effective, possibly admissible heuristic $h(n)$.

1.5 Tree Search vs. Graph Search

The problem space solved by A* is fundamentally a graph, yet the search strategy can be executed either as a Tree Search or a Graph Search. The difference lies entirely in how the algorithm handles states (nodes) that can be reached via multiple distinct paths, a common occurrence in graphs with cycles or multiple connecting routes.

1.5.1 Tree Search A*

The Tree Search variant treats every path to a state as a unique node in the search tree.

- **Mechanism:** Does not utilize a Closed Set. A node is generated and expanded without checking if an identical state has been reached previously via a different path.
- **Efficiency:** Avoiding the memory overhead of the Closed Set, leads to the redundant expansion of identical states, severely impacting time efficiency, especially in graphs with many cycles or alternative routes.
- **Optimality:** Only guaranteed if and only if the heuristic function $h(n)$ is admissible.

1.5.2 Graph Search A*

The Graph Search variant is the standard implementation of A* because it explicitly tracks visited states to avoid redundant work.

- **Mechanism:** Utilizes both the Open Set and the Closed Set to ensure that, ideally, no state is expanded more than once.
- **Efficiency:** Significantly more time-efficient than Tree Search, as it avoids re-exploring entire subgraphs.
- **Optimality:**
  - If the heuristic $h(n)$ is **consistent**, a node is guaranteed to be reached via the optimal path when it is first placed in the Closed Set. Therefore, no node ever needs to be re-opened.
  - If $h(n)$ is only **admissible**, the optimal path to a state might be discovered after the state has already been placed in the Closed Set. This motivates the re-opening mechanism.

1.5.3 Implementation Choice

In the context of this project, which adapts A* for machine learning optimization, given the experimental nature of the heuristic $h(n)$ and the $g(n)$ cost definition based on training loss, neither strict **admissibility** nor **consistency** can be guaranteed. Therefore, the implementation utilizes the Graph Search A* algorithm, characterized by the explicit use of both the Open Set and the Closed Set. Crucially, the algorithm includes the necessary logic to re-open nodes from the Closed Set if a new, cheaper path to that node is discovered. This ensures the best possible path is found despite the non-ideal properties of the custom heuristic.

Chapter 2

The Multilayer Perceptron and Traditional Training Methods

2.1 Multilayer Perceptron (MLP) Architecture

The Multilayer Perceptron (MLP) is a fundamental class of feed-forward artificial neural networks. It is distinguished by having one or more *hidden layers* between the input layer and the output layer, which allows the network to learn complex, non-linear relationships between inputs and outputs.

2.1.1 Neuron Structure and Function

The basic unit of the MLP is the artificial neuron, which receives a set of inputs, each associated with a synaptic weight. The neuron performs two main operations:

1. **Weighted Aggregation:** It calculates the weighted sum of the inputs, to which a *bias* (β) is added:

   $$z = \left(\sum_{i=1}^{m} w_i x_i \right) + \beta$$

   where x_i are the inputs, w_i are the weights, and m is the number of inputs.

2. **Activation Function:** It applies a non-linear activation function (σ) to the aggregated result to produce the neuron's output (a):

   $$a = \sigma(z)$$

2.1.2 The Cost Function

Training an MLP is an optimization problem that aims to minimize the error between the network's predicted output and the desired target output. This error is quantified by the *cost function* (or loss function).

2.2 The Standard Training Method: Backpropagation

Traditionally, the MLP is trained using the **Backpropagation** (error retro-propagation) algorithm, which is based on the *Gradient Descent* method.

2.2.1 Gradient Descent

Gradient Descent is an iterative algorithm that adjusts the network's weights to move in the direction where the cost function decreases most rapidly. The weight update rule is given by:

$$w_{i,\text{new}} = w_{i,\text{old}} - \eta \frac{\partial E}{\partial w_i}$$

where η is the *learning rate* and $\frac{\partial E}{\partial w_i}$ is the gradient, which is the partial derivative of the error with respect to the weight w_i.

2.2.2 Error Backpropagation

Backpropagation is the efficient mechanism used to calculate this gradient. It utilizes the chain rule of differential calculus, propagating the error (and thus the gradient) backward, from the output layer to the hidden layers and finally to the input layer. This allows for the determination of the exact contribution of each weight to the total error, making the optimization scalable for deep networks.

2.3 Motivation for the A* Approach

Despite its effectiveness, Backpropagation has several critical limitations that motivate the exploration of alternative optimization methods, such as A:

- **Local Minima:** Gradient Descent tends to get trapped in local minima or flat regions (plateaus) of the cost surface, preventing the network from achieving the global optimal solution.

- **Parameter Sensitivity:** The performance of Backpropagation is highly dependent on the correct selection of the η hyperparameter (learning rate). A value that is too high can cause oscillations, while a value that is too low makes training excessively slow.

- **Local Nature:** Backpropagation is a purely local algorithm, guided only by the gradient information at the current point. It lacks the global "vision" of the cost surface that an informed search algorithm like A* possesses.

This report proposes to frame the MLP training not as a continuous descent problem, but as a pathfinding problem in the quantize weight space, where A* can potentially overcome the local exploration limitations of Backpropagation, albeit at the expense of algorithmic training efficiency.

Chapter 3

The A* Search Framework for MLP Training

The primary objective of this project is to reformulate the optimization challenge of training a Multilayer Perceptron (MLP) as a **shortest-path problem** within a discrete, high-dimensional weight space. This chapter introduces the framework designed to achieve this, detailing how the continuous weight space is discretized, how the nodes and edges of the search graph are defined, and how the A* algorithm's evaluation function is adapted to guide the network towards optimal weights.

3.1 Discretization of the Weight Space

The A* algorithm fundamentally operates on a discrete graph. Therefore, the continuous domain of real-valued weight parameters must be converted into a finite, discrete set of possible states. This is handled by the `QuantizedMLP` class.

3.1.1 Quantized State Definition

To define the search nodes, the floating-point parameters (weights and biases) of the MLP are subjected to uniform quantization. Given a predefined **quantization factor** (Q), every weight w_i is rounded to the nearest multiple of $1/Q$.

$$w_{i,\text{quantized}} = \text{round}(w_i \cdot Q)/Q$$

The number of potential discrete weight configurations defines the size of the search space. To manage the complexity and avoid instability, the parameters are constrained to a defined numerical range $[\text{min\_range}, \text{max\_range}]$.

The total number of possible states is:

$$[\mathrm{Q}(\mathrm{min\_range} + \mathrm{max\_range}) + 1]^{N_{\mathrm{params}}}$$

3.1.2 State Representation and Hashing

In the A* implementation, the efficacy of the algorithm relies on quickly checking if a particular weight configuration has been visited before, or if a better path to that configuration has been found. This requires a unique, hashable identifier for each state.

The `QuantizedMLP` utilizes the `get_state_hash()` method, which concatenates all quantized parameters into a single, ordered tuple of integers (by multiplying the quantized weights by the quantization factor Q). This tuple serves as the key in the Closed Set (represented by the `g_costs` dictionary in the implementation) for tracking the minimum cost found so far to reach that state.

3.2 Search Node and Transition Definition

3.2.1 The Search Node

A single search node n is represented by the `SearchNode` class and contains:

1. The current `QuantizedMLP` configuration (a unique, quantized set of weights).
2. The calculated cost-to-come, $g(n)$.
3. The heuristic estimate to the goal, $h(n)$.
4. The total estimated cost, $f(n) = g(n) + h(n)$.

3.2.2 Defining Transitions (Neighbors)

The edges of the search graph are defined by the allowed transitions between weight configurations, generated by the `get_neighbors` function. In this structured approach, a transition represents a modification to a localized block of parameters rather than a single disjointed weight.

A neighbor state s is generated from the current state n by applying a sliding window of size $H \times W$ for weight matrices (or a corresponding 1D window for bias vectors) across the layers. Every parameter $w_{i,j}$ falling within the boundaries of the window at a specific position p is updated by exactly one quantization step:

$$w_{i,j,s} = w_{i,j,n} \pm \frac{1}{Q} \quad \forall (i,j) \in \text{Window}_p$$

The number of possible transitions (neighbors) per layer is determined by the total number of valid window positions in a matrix of size $M \times N$, given a horizontal stride x_s and vertical stride y_s:

$$\text{Slides} = \left(\lfloor \frac{M - H}{y_s} \rfloor + 1 \right) \times \left(\lfloor \frac{N - W}{x_s} \rfloor + 1 \right)$$

This structured transition model is adopted to provide several key benefits over traditional single-scalar modifications:

- **Exploitation of Spatial Locality:** By updating clusters of weights simultaneously, the search effectively navigates the functional block nature of neural network layers. This aligns with the intuition that neighboring weights often contribute to the same local feature representations.
- **Search Space Pruning:** Transitioning via kernels significantly reduces the branching factor of the search graph. This pruning of possible next states allows the algorithm to focus on regional adjustments that have a more cohesive impact on the network's output.
- **Computational Efficiency:** The structured approach leads to a substantial decrease in training time and memory consumption. By generating fewer but more meaningful neighbor states, the method minimizes the memory overhead typically associated with tracking massive neighborhood sets.
- **Scalability:** These performance gains are critical for applying the methodology to larger and deeper network architectures, effectively mitigating the curse of dimensionality inherent in high-dimensional weight spaces.

The total number of neighbors for any given state n is thus:

$$N_{\text{neighbors}} = 2 \cdot \sum_{\ell \in \text{Layers}} \text{Slides}_\ell$$

ensuring a controlled and computationally feasible traversal of the weight space.

$$W^{(l)} = \begin{bmatrix} w_{1,1}^{(l)} & w_{1,2}^{(l)} & w_{1,3}^{(l)} & w_{1,4}^{(l)} & w_{1,5}^{(l)} \\ w_{2,1}^{(l)} & w_{2,2}^{(l)} & w_{2,3}^{(l)} & w_{2,4}^{(l)} & w_{2,5}^{(l)} \\ w_{3,1}^{(l)} & w_{3,2}^{(l)} & w_{3,3}^{(l)} & w_{3,4}^{(l)} & w_{3,5}^{(l)} \\ w_{4,1}^{(l)} & w_{4,2}^{(l)} & w_{4,3}^{(l)} & w_{4,4}^{(l)} & w_{4,5}^{(l)} \end{bmatrix}$$

Figure 3.1: Plot of the sliding window in position (1,1)

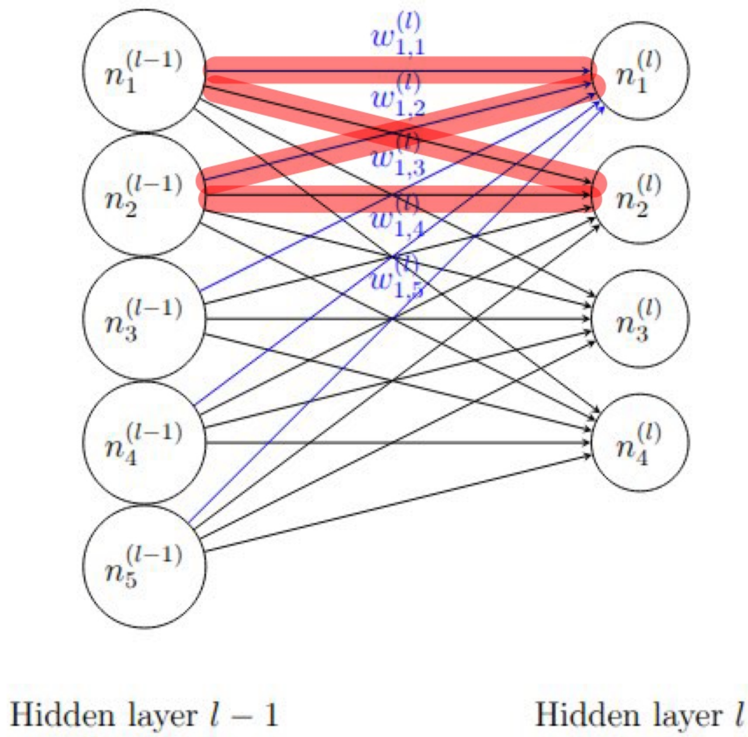

Image representing all the connections (blue) between the first neuron of layer l and all the neurons of the previous layer

Figure 3.2: Plot of the connection involved in the sliding window positioned in (1,1)

3.3 A* Cost Function Adaptation for Training

The successful application of A* to optimization relies entirely on correctly defining the cost components $g(n)$ and $h(n)$ in the context of neural network training.

3.3.1 Heuristic Function $h(n)$: Current State Loss

In this formulation, the heuristic $h(n)$ directly represents the network's performance at node n and it is defined as the current loss value evaluated on the training data:

$$h(n) = L(n)$$

This heuristic prioritizes configurations that minimize the objective function, guiding the search toward lower-cost states in the weight landscape.

3.3.2 Path Cost $g(n)$: Accumulated Loss Differential

The cost-to-come, $g(n)$, represents the cumulative cost of the path taken from the initial configuration to the current node n. The cost of each transition (edge) is defined by the improvement or degradation in loss between the parent and the child node n.

The step cost Δg is defined as:

$$\Delta g = L(\text{n}) - L(\text{parent})$$

The total path cost is then updated iteratively:

$$g(n) = g(\text{parent}) + \Delta g$$

By defining Δg in terms of the loss differential, the path cost effectively tracks the total variation in loss encountered during the traversal of the search space.

3.3.3 Total Evaluation Function $f(n)$

The combined evaluation function $f(n)$ determines the priority of nodes in the search queue:

$$f(n) = g(n) + h(n) = (g(\text{parent}) + (L(n) - L(\text{parent}))) + L(n)$$

By selecting the node with the minimum $f(n)$, the algorithm seeks a path that minimizes both the absolute error of the current state and the cumulative change in loss required to reach that state.

3.3.4 Admissibility Analysis

In standard A* search, a heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, i.e., $h(n) \leq h^*(n)$. In this revised framework, the "goal" is implicitly the global minimum of the loss landscape ($L = 0$), and the cost is measured in units of loss differential.

1. **Definition Alignment:** Both the heuristic $h(n) = L(n)$ and the true cost $h^*(n)$ are expressed in the same units (loss). The true cost $h^*(n)$ represents the cumulative sum of loss differentials required to reach the global minimum.

2. **The Admissibility Challenge:** For $h(n)$ to be admissible, the condition $h(n) \leq h^*(n)$ must hold. Under the current definitions, $h(n) = L(n)$, which is a non-negative value (the loss), while the true cost $h^*(n)$ is the sum of loss differentials: $h^*(n) = \sum \Delta g$.

   In any scenario where the path toward the goal is monotonically improving, every step cost $\Delta g = L(n') - L(n)$ is strictly negative. Consequently, the true cost-to-go $h^*(n)$ becomes a summation of negative terms, yielding a negative total value. Because $L(n) \geq 0$, the inequality $L(n) \leq h^*(n)$ is fundamentally violated in every improving step of the training process.

   This implies that the heuristic is *never admissible* during a standard training descent. It provides a positive value (L) for a trajectory that accumulation a negative total cost ($\sum \Delta g$). Admissibility could only be restored if the search moved "uphill" into regions of significantly higher loss; however, since the primary goal is loss minimization, this violation is a permanent structural feature of the search design rather than an error.

3.3.5 Consistency Analysis

A heuristic is consistent if $h(n) \leq c(n, n') + h(n')$, where $c(n, n')$ is the cost of the transition. With our new definitions:

- $h(n) = L(n)$
- $c(n, n') = \Delta g = L(n') - L(n)$

- $h(n') = L(n')$

Substituting these into the consistency inequality:

$$L(n) \leq (L(n') - L(n)) + L(n') \implies L(n) \leq 2L(n') - L(n) \implies 2L(n) \leq 2L(n')$$

Simplifying this, the **Consistency Condition** becomes:

$$L(n) \leq L(n')$$

Analysis of Scenarios w.r.t. Consistency

Unlike the previous version, consistency is now tied directly to the direction of the loss change ΔL:

1. **Scenario 1: Loss Decrease ($\Delta L < 0$):** When the sliding window update improves the model, $L(n') < L(n)$. In this case, the consistency condition $L(n) \leq L(n')$ is **violated**. This is a mathematically interesting result: every "good" step in training (one that reduces loss) technically violates the A* consistency property under these definitions.

2. **Scenario 2: Loss Increase ($\Delta L > 0$):** If the update moves into a worse region of the landscape, $L(n') > L(n)$. Here, the consistency condition is **satisfied**. This implies that the search is only "consistent" when it is moving away from the solution.

3. **Scenario 3: Stationary Loss ($\Delta L = 0$):** In plateaus where $L(n') = L(n)$, the heuristic is perfectly consistent.

3.3.6 Efficiency and Scalability of Kernel-Based Transitions

The implementation of sliding window kernels, paired with the loss-differential cost function, provides a robust framework for navigating high-dimensional weight spaces. This structured approach ensures the search remains computationally feasible through several key mechanisms:

- **Memory Optimization:** By generating transitions based on window "hops", the algorithm significantly restricts the branching factor of the search graph. This keeps the A* Open List manageable and prevents memory exhaustion, even when applied to larger Multi-Layer Perceptron (MLP) architectures.

- **Temporal Acceleration:** Each kernel application updates a block of $H \times W$ parameters simultaneously. This spatial grouping effectively "squashes" the required search depth, allowing the algorithm to achieve significant loss reduction in fewer state explorations compared to unstructured methods.

By leveraging these efficiencies, the kernel-based method mitigates the "curse of dimensionality," making it possible to apply discrete search-based optimization to complex neural network structures that would otherwise be computationally prohibitive.

3.4 The A* Training Loop Implementation

The core training logic is contained within the `Trainer` class, which manages the A* search:

1. **Initialization:** The process begins by creating an `initial_node` from the starting MLP weights. This node is placed in the priority queue (`open_set`).
2. **Iteration:** In each iteration, the node with the lowest $f(n)$ is retrieved from the `open_set`.
3. **Optimality Check:** Before expanding the node, the current path cost $g(n)$ is checked against the minimum cost previously recorded for that state in `g_costs` to ensure that A* always processes the optimal path to a given state first.
4. **Expansion:** Neighbors are generated by the transition rule ($\pm 1/Q$ change to a weight subset).
5. **Cost Update:** For each neighbor, the new g and h values are calculated. If the new path to the neighbor results in a lower cost-to-come ($g_{\text{new}} < g_{\text{old}}$), the neighbor is either updated in the search space or added to the `open_set`.
6. **Termination:** The search terminates when either the best node retrieved from the `open_set` satisfies the goal condition ($h(n) = 0$) or the maximum number of iterations is reached.

This framework provides a global, informed search mechanism for MLP training, aiming to overcome the local minimum issues inherent in traditional gradient-based methods like Backpropagation.

Chapter 4

Experimental Methodology and Results

This report conducts a comparative analysis of two optimization methodologies, focusing on their respective strengths and weaknesses.

The primary objective of this study is to analyze a novel approach using A* search principles, experimentally comparing its performance and behavior against the well-established optimization methodology of Gradient Descent. Furthermore, a crucial element of this comparison involves a detailed investigation into the A* framework's sensitivity to its core parameters; a systematic hyperparameter finetuning process has been performed to understand their influence on achieving superior performance.

4.1 Experimental Methodology

To systematically evaluate the proposed A* search framework, a three-stage comparative protocol was designed and applied uniformly to sine dataset. This approach allows for a rigorous analysis of the A* algorithm's behavior under different hyperparameter settings and a direct performance comparison with a conventional gradient descent method.

4.1.1 MLP Architecture and Evaluation

For all experiments, a Multilayer Perceptron (MLP) architecture suitable for the complexity of the dataset was chosen. The loss function $L(n)$ for both A* evaluation (heuristic $h(n)$) and the Backpropagation baseline was chosen based on the task.

4.1.2 Three-Stage Comparative Protocol

Stage 1: Hyperparameter Grid Search and Sensitivity Analysis

The A* search performance is critically dependent on the discretization of the weight space. Therefore, the first stage involved an exhaustive grid search to:

1. Identify the optimal configuration of parameters that maximizes convergence speed and solution quality.

2. Understand the sensitivity of the A* search to changes in the search space.

The key hyperparameters subjected to optimization were:

- **Maximum Iterations (max_iterations):** The upper limit of the number of total node expansions.

- **Quantization Factor (Q):** Defining the granularity of the weight space ($1/Q$).

- **Parameter Range:** The boundary $[\text{min\_range}, \text{max\_range}]$ for the weights.

- **Kernels Shape and Strides:** This parameter set defines the window dimensions ($H \times W$) for the weight and bias matrices, alongside the horizontal (x_s) and vertical (y_s) strides.

Stage 2: A* Training with Optimal Configuration

The best-performing hyperparameter set from Stage 1 was selected and used to train the final MLP model. This training run recorded the total number of node expansions, the path taken (sequence of weight states), and the overall wall-clock time required for the optimization.

Stage 3: Backpropagation Baseline and Comparison

The A* performance was compared against the standard Stochastic Gradient Descent algorithm with Backpropagation. For a fair comparison, the SGD baseline was trained using its own optimal learning rate (η) derived from preliminary tuning. The final comparison focused on three critical metrics:

- **Convergence Speed:** Total A* node expansions vs. total SGD epochs required to achieve the minimum loss.

- **Final Loss Value:** The best loss achieved during training for both methods.
- **Wall-Clock Time:** Comparison of total training time.

4.2 Synthetic Dataset: Sine Function Regression

The first experiment applies the A* training framework to a non-linear regression problem, assessing its ability to learn complex continuous functions.

4.2.1 Dataset Generation and Objective

The synthetic dataset was generated using the one-dimensional sine function with additive Gaussian noise following the Normal distribution.

- **Input (x):** 1,000 evenly spaced samples in the range $x \in [0, 4\pi]$.
- **Output (y):** Generated as $y = \sin(x) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 0.1)$.
- **MLP Architecture:** Input (1 neuron) $\rightarrow$ Hidden Layer 1 (8 neurons, $ReLU$) $\rightarrow$ Hidden Layer 2 (16 neurons, $ReLU$) $\rightarrow$ Hidden Layer 3 (8 neurons, $ReLU$) $\rightarrow$ Output (1 neuron, $tanh$).

The network's task is to learn the underlying $\sin(x)$ mapping, minimizing the Mean Squared Error loss.

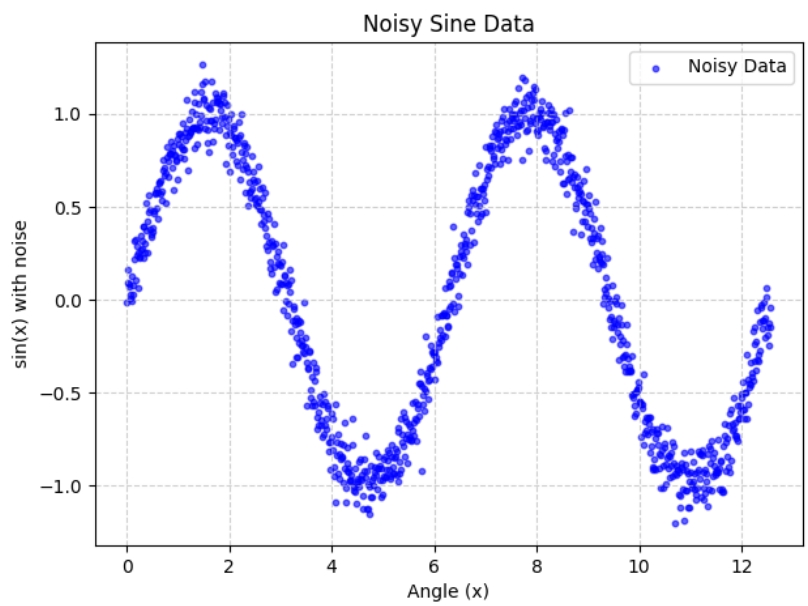

Figure 4.1: Plot of the noisy sine dataset

4.2.2 Stage 1: Grid Search Findings

Except for the specific parameter under evaluation during the grid search, the system is initialized with the following default hyperparameters:

- **Quantization Factor (Q):** 10
- **Parameter Range:** $[-20, 20]$
- **Kernel Geometry:** Weight kernels of size 3×3 and bias kernels of size 3.
- **Traversal Strides:** Horizontal (x_s) and vertical (y_s) strides are both set to 2.
- **Search Constraints:** A maximum of 2000 iterations.

Impact of the Quantization Factor
Based on the empirical results presented in the accompanying figure, the hyperparameter grid search demonstrated a clear dependency of the final achieved loss on the selected quantization factor, Q. Specifically, among the tested values ($Q = 10$, $Q = 100$, and $Q = 1000$), the lower and intermediate factors of $Q = 10$ and $Q = 100$ yielded the most favorable and comparable outcomes, both achieving a final mean loss of approximately 0.17.

In contrast, the highest quantization factor, $Q = 1000$, resulted in significantly poorer performance, with a final mean loss of 0.41. This suggests that while $Q = 10$ and $Q = 100$ provide a sufficiently discretized weight space for efficient A* traversal, $Q = 1000$ leads to a search space that is too fine-grained. At this higher resolution, the distance between meaningful states may hinder the algorithm's ability to reach a global minimum within the allocated iteration budget.

To evaluate the stability and convergence of the proposed methodology, the following results represent the average loss across 10 independent runs for each tested hyperparameter configuration. Figure below provides a clear measure of statistical variability and reproducibility, where the solid lines represent the average loss and the surrounding shaded regions indicate the one standard deviation ($\pm 1\sigma$) interval.

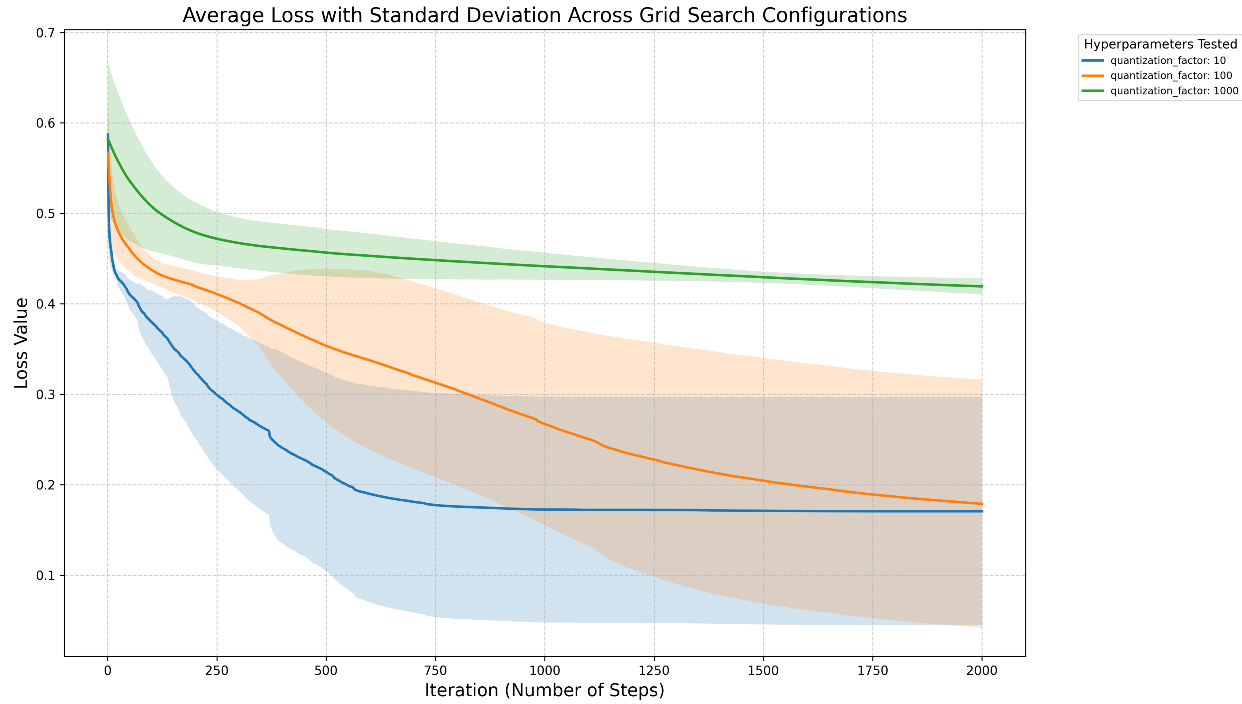

Figure 4.2: Comparison of the mean loss across training iterations for varying quantization factors ($Q \in \{10, 100, 1000\}$)

Table 4.1: Impact of Quantization Factor (Q) on Sine Regression Performance

QF (Q)	Mean Loss	Median Loss	Std. Dev.	Min Loss	Max Loss	Mean Time (s)
10	**0.1705**	**0.1049**	0.1261	0.0570	0.4328	**367.11**
100	0.1788	0.1093	0.1373	0.0435	0.3834	423.68
1000	0.4192	0.4222	0.0088	0.4018	0.4304	431.09

Impact of the Network Parameters Range

A second set of experiments was conducted to determine the optimal numerical weight range for the network parameters, defining the boundary constraints for the quantized weights. We evaluated three distinct intervals: $[-5, 5]$, $[-10, 10]$, and $[-20, 20]$.

Analysis of the results demonstrates a non-monotonic trend in performance relative to the range width. The intermediate range of $[-10, 10]$ yielded the best performance, achieving the lowest mean loss of 0.1672. In comparison, the narrowest range ($[-5, 5]$) resulted in a higher mean loss of 0.2507, likely due to insufficient model expressiveness. Interestingly, the widest range ($[-20, 20]$) also showed a performance degradation compared to the medium interval, with a mean loss of 0.2100.

These observations suggest that while a range must be broad enough to provide the necessary search capacity, an excessively wide weight space may increase the search complexity. Consequently, the $[-10, 10]$ interval strikes the most effective balance between parameter flexibility and search efficiency for this architecture.

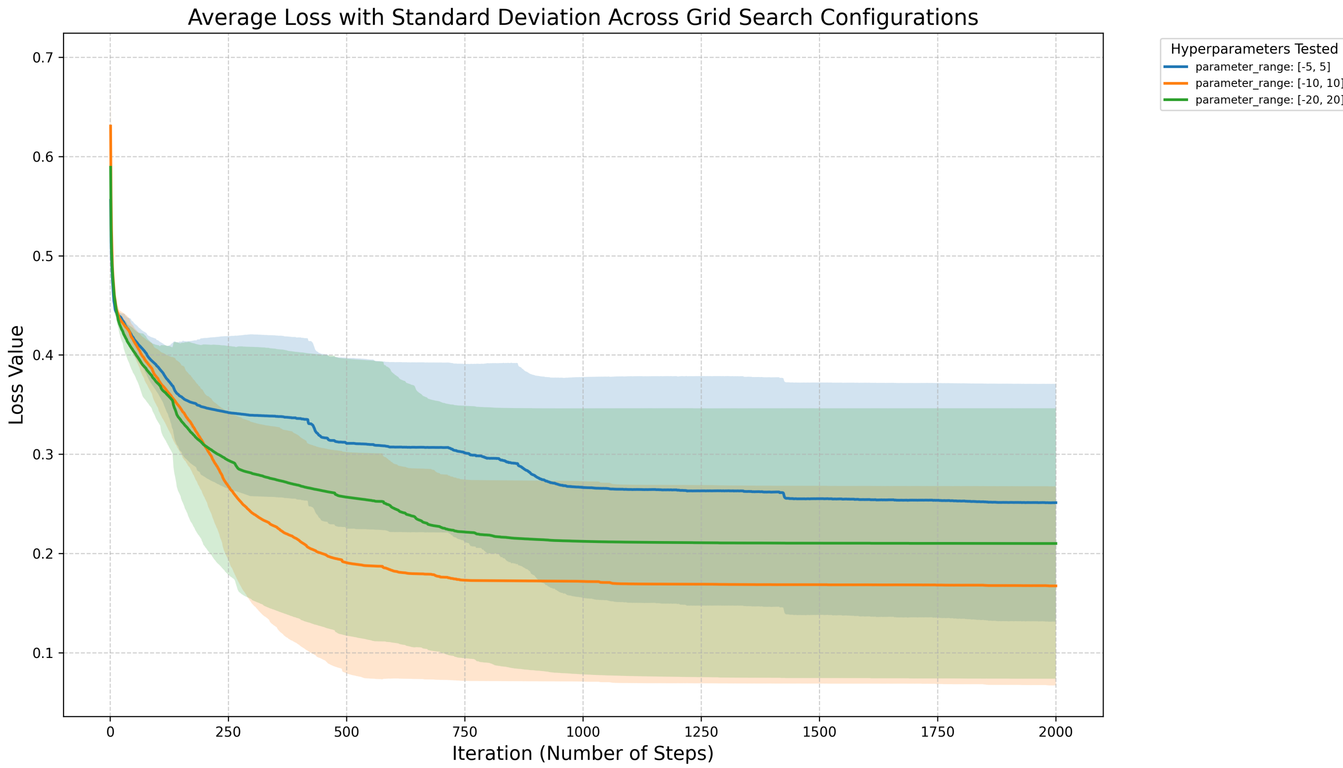

Figure 4.3: Comparison of the mean loss across training iterations for varying Parameter Ranges

Table 4.2: Impact of Parameter Range (PR) on Sine Regression Performance

PR	**Mean Loss**	**Median Loss**	**Std. Dev.**	**Min Loss**	**Max Loss**	**Mean Time (s)**
$[-5, 5]$	0.2507	0.2666	0.1200	0.0599	0.4259	274.99
[-10, 10]	**0.1672**	**0.1456**	0.1003	0.0618	0.3893	312.69
$[-20, 20]$	0.2100	0.2104	0.1363	0.0433	0.3717	369.17

Effect of Maximum Iterations ($I_{\max}$)

The third parameter evaluated was the maximum number of search iterations ($I_{\max}$), testing values of $1,000$ and $2,000$. Unlike fixed-step methodologies, in this framework, $I_{\max}$ acts as a hard cap on the exploration budget within the weight space.

As demonstrated by the empirical results, the configuration with $I_{\max} = 1,000$ iterations proved to be the most efficient, achieving a mean terminal loss of 0.1789. Interestingly, doubling the computational budget to $I_{\max} = 2,000$ did not yield a performance gain, instead resulting in a slightly higher mean loss of 0.1884 and a significant increase in average execution time from approximately 201 seconds to 348 seconds.

This observation suggests that the search typically converges to a stable local minimum or a satisfactory solution within the first $1,000$ iterations. Rather than facilitating the discovery of deeper minima, the extended budget allows the algorithm to persist in high-dimensional plateaus or oscillate in local regions without meaningful improvement.

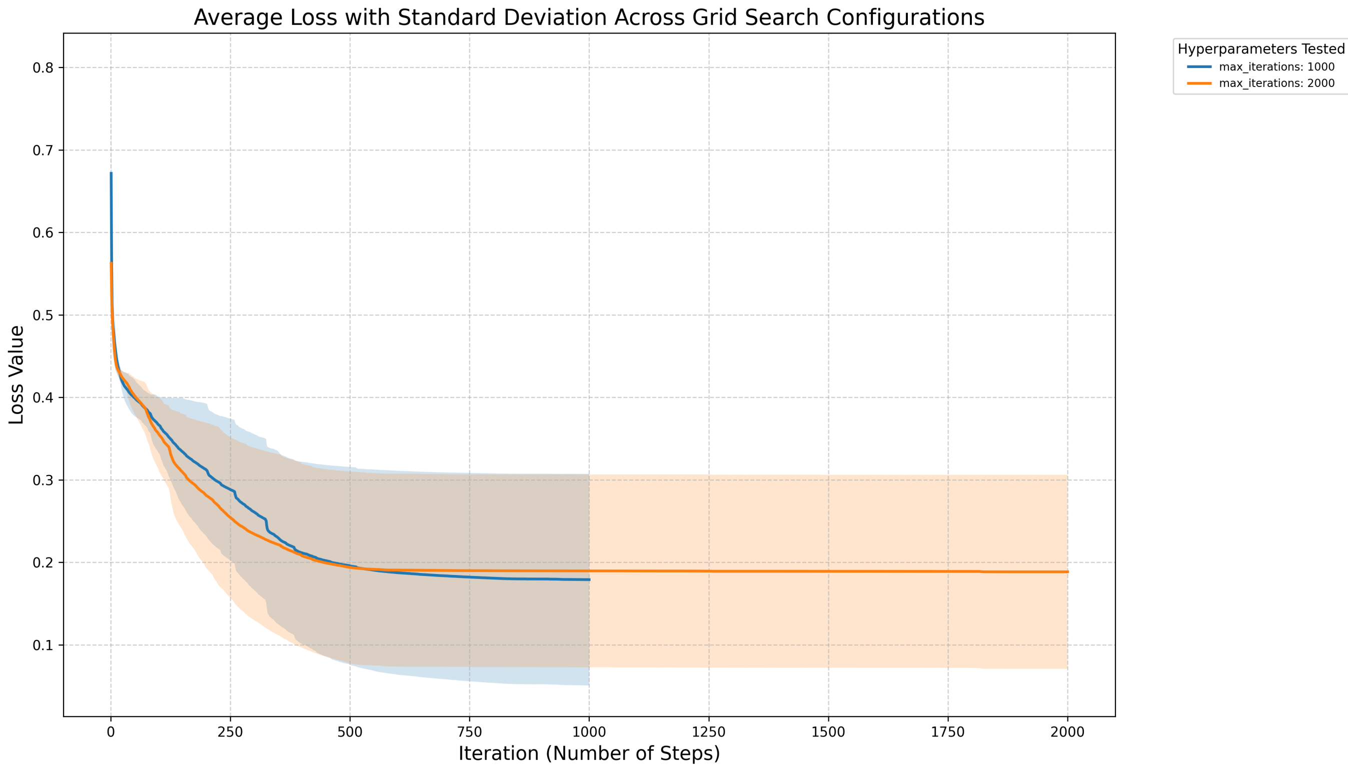

Figure 4.4: Comparison of the mean loss across training iterations for varying $I_{\max}$ parameter

Table 4.3: Impact of Maximum Iterations on Sine Regression Performance

Iterations	**Mean Loss**	**Median Loss**	**Std. Dev.**	**Min Loss**	**Max Loss**	**Mean Time (s)**
1000	**0.1789**	**0.1360**	0.1283	0.0380	0.3563	**201.29**
2000	0.1884	0.1591	0.1176	0.0566	0.3546	347.74

Effect of Kernel Geometry and Traversal Strides
A critical set of experiments was conducted to evaluate the influence of the sliding window geometry on search efficiency. We tested four configurations varying the Weight Kernel size (WK), Bias Kernel size (BK), and the uniform traversal stride (S), where horizontal and vertical strides were kept equal ($x_s = y_s = S$). These parameters determine the search graph's branching factor and the 'spatial span' of each transition, effectively governing the extent of weight space coverage per exploration step.
Analysis of the results indicates that the choice of kernel dimensions significantly impacts the convergence rate and terminal loss. Larger kernels effectively "squash" the search depth by updating more parameters per hop, while the stride regulates the overlap between successive updates.

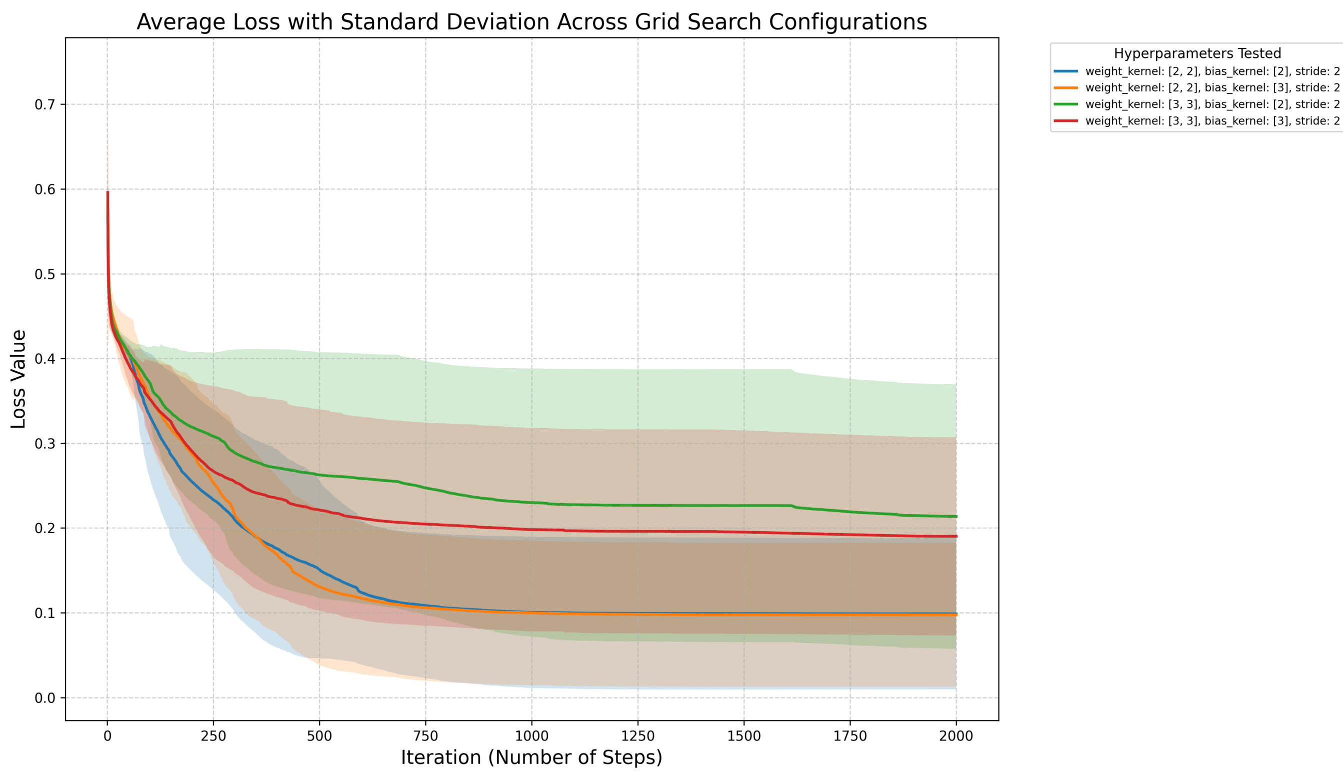

Figure 4.5: Evolution of mean loss across iterations for varying kernel geometries

Table 4.4: Impact of Kernel Sizes and Strides on Sine Regression Performance (10 Repetitions)

WK	BK	S	Mean Loss	Median Loss	Std. Dev.	Min Loss	Max Loss	Mean Time (s)
(2, 2)	(2)	2	0.0987	0.0565	0.0892	0.0196	0.3134	499.99
(2, 2)	(3)	2	**0.0975**	0.0673	0.0848	0.0217	0.3199	478.15
(3, 3)	(2)	2	0.2136	0.1644	0.1560	0.0505	0.4360	395.87
(3, 3)	(3)	2	0.1902	0.1706	0.1168	0.0673	0.3973	**357.17**

This structured traversal ensures that the search focuses on localized functional blocks, prioritizing regional weight adjustments. By optimizing these geometric parameters, the algorithm achieves a balance between granular precision and broad exploration, mitigating the computational overhead typically associated with discrete search in high-dimensional spaces.

4.2.3 Stage 2 & 3: Training and Comparative Analysis

Explanation of Plots: Statistical Methodology
The comparative analysis of A* search and Gradient Descent relies on two complementary visualization techniques, each providing a distinct view of the algorithms' performance across the ten independent runs.
Mean Loss with Standard Deviation Shading
The solid line represents the average loss ($\bar{L}$) across all ten runs at every iteration, clearly demonstrating which method achieves faster convergence or reaches a lower loss plateau on average.
Interpretation of Variability (Standard Deviation $\pm 1\sigma$): The shaded area surrounding the mean line represents the standard deviation ($\pm 1\sigma$) of the loss at each iteration. This is a crucial measure of an algorithm's consistency and robustness.

- A narrow shaded band means the method is highly consistent, its performance is reliable and typically near the average, regardless of the random initial weights.
- A wide shaded band means the method is volatile (σ is large) and highly dependent on the random initialization. A wide band often signals that the algorithm frequently gets stuck in poor local minima or conversely, occasionally finds good solutions.

Final Loss Box Plot
This plot provides a concise summary of the distribution of the final terminal loss achieved by each method after the completion of $I_{\max}$ iterations. It is ideal for comparing the final state quality.
Interpretation of Distribution:

- **The Median Line:** The horizontal line inside the box shows the median (the 50th percentile) result. The median is a robust measure of typical performance, as it is less sensitive to extreme outliers than the mean.

- **The Box (Interquartile Range, IQR):** The vertical span of the box represents the middle 50% of your results (from the 25th to the 75th percentile). A very short box indicates high clustering and consistency in the results. If one box is much lower than another, that method is definitively superior in its typical performance range.

- **Whiskers and Dots (Range and Outliers):** The whiskers show the full range of results (excluding outliers), and individual dots beyond the whiskers identify extreme outliers. These elements help to see how often a method results in a failed run or a single exceptional result.

Calculation of Whisker Range and Outliers
The length of the whiskers is not arbitrary; it is statistically determined by the data's central spread (the IQR) to identify extreme values (outliers).

- **Interquartile Range (IQR):** This is the length of the box, calculated as $IQR = Q_3 - Q_1$.

- **Whisker Boundaries:** The upper and lower whiskers extend to the most extreme data points that are no more than $\mathbf{1.5 \times IQR}$ away from the box edges. The boundaries are formally defined as:

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR$$

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR$$

- **Outliers:** Any final loss value that falls outside these calculated bounds is considered an outlier and is plotted as an individual point (dot). In the context of loss, points above the upper whisker represent runs where the algorithm performed significantly worse than its typical operation.

4.2.4 Comparison with Gradient Descent: $I_{\mathbf{max}} = 1,000$

To quantitatively assess the performance of the A* search approach against the established optimization method of Gradient Descent (GD) on the sine dataset, a comparative study was conducted. Both A* and GD were executed for $I_{\max} = 1,000$ training steps. To ensure robustness, ten independent runs ($n = 10$) were performed for each algorithm, starting from different random weight initializations. The statistical results are summarized in Table 4.5, and the performance trajectory is presented in the following figures.

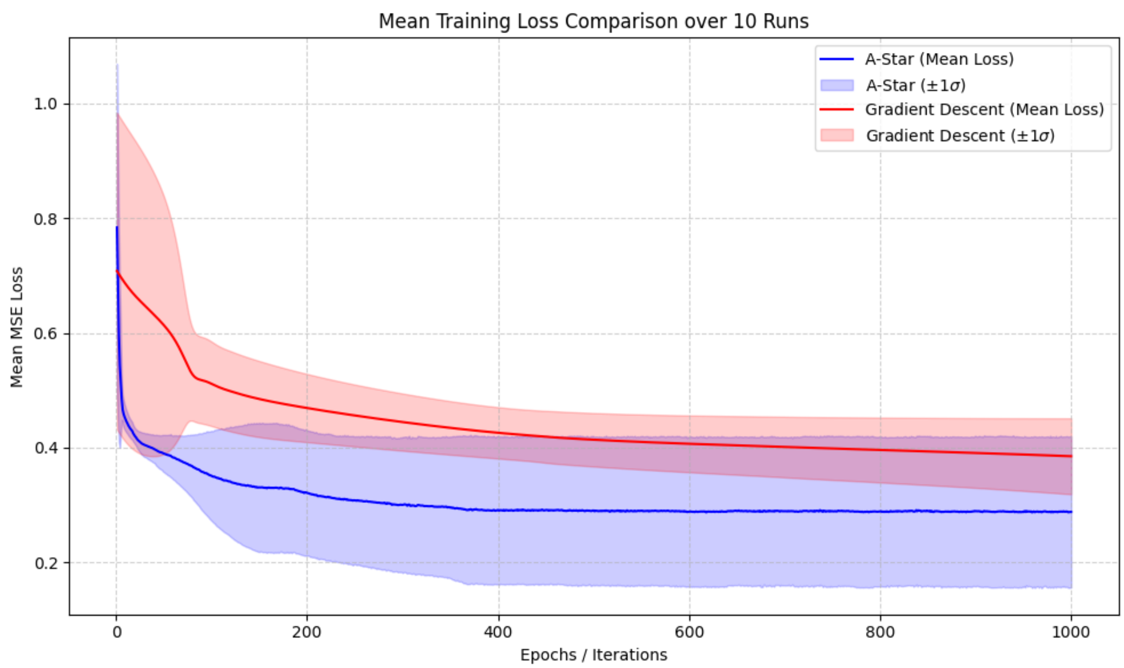

Figure 4.6: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 1{,}000$ Iterations. This plot illustrates the average loss over ten independent runs for both A* Search (Novel) and Gradient Descent (Classic), with the shaded region representing ±1 standard deviation. It highlights convergence speed and the consistency of each algorithm's performance over time.

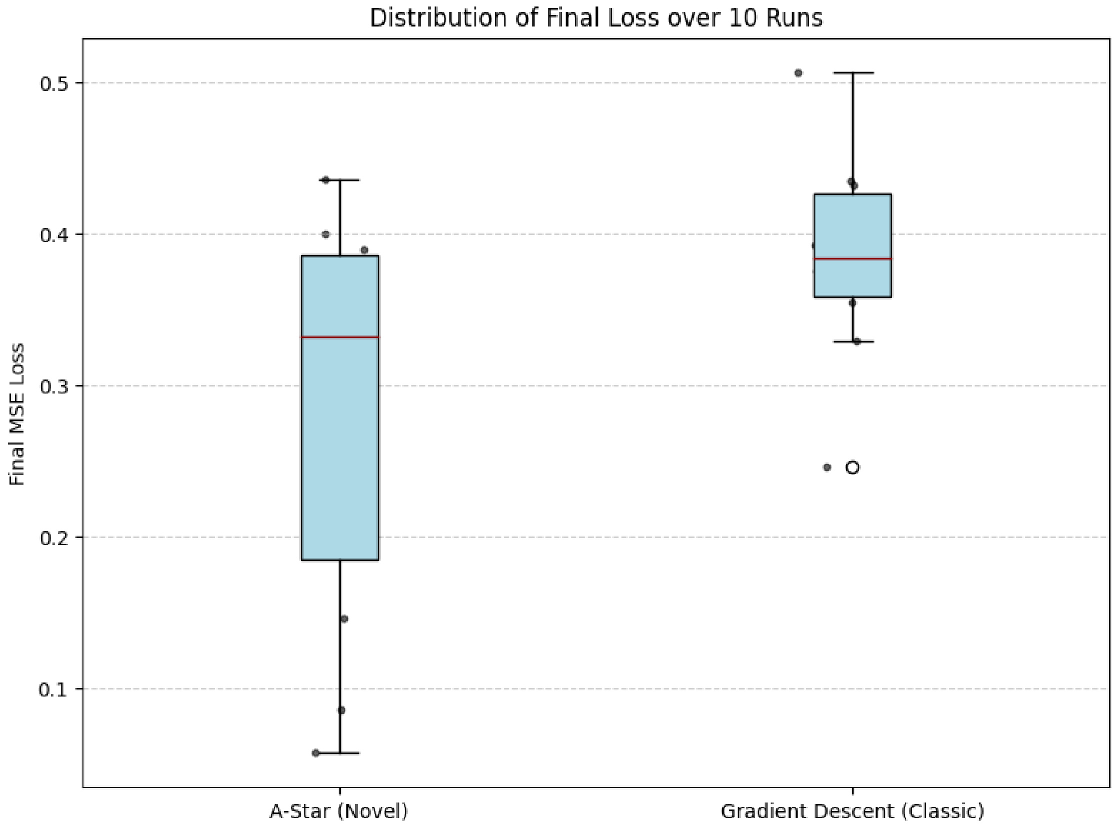

Figure 4.7: Final Loss Box Plot for $I_{\max} = 1{,}000$ Iterations. This box plot summarizes the distribution of the terminal loss values achieved by A* Search (Novel) and Gradient Descent (Classic) over ten independent runs. The box represents the interquartile range (IQR), the central line indicates the median, and whiskers show the range of results (excluding outliers).

Table 4.5: Comparative Statistics Over Ten Runs for $I_{\max} = 1,000$ Iterations

Metric	**A* Search (Novel)**	**Gradient Descent (Classic)**
Average Loss ($\bar{L}$)	**0.285213**	0.385243
Median Loss	**0.331870**	0.384283
Standard Deviation (σ)	0.130934	**0.066165**
Variance (σ^2)	0.017144	**0.004378**
Minimum Loss ($L_{\min}$)	**0.057494**	0.246618
Maximum Loss ($L_{\max}$)	**0.436191**	0.506505
Average Training Time (s)	100.10	**10.20**

Statistical and Trajectory Analysis

Performance (Average Loss): For the 1,000-iteration budget, the A* search method significantly outperformed GD in terms of central tendency. The A* Average Loss ($\bar{L} = 0.2852$) is substantially lower than the GD Average Loss ($\bar{L} = 0.3852$), a difference of nearly 0.1 units. This is further supported by the Median Loss, where A* is also clearly superior, indicating that A* typically converges to a better solution within this short number of search iteration.
Best-Case Potential: A* demonstrated a superior capacity to locate deep minima, achieving a Minimum Loss ($L_{\min} = 0.0575$) that is four times better than GD's best run ($L_{\min} = 0.2466$). This highlights A*'s effective, albeit non-admissible, exploration strategy in the quantized space.
Consistency Trade-off: The major point of contrast is in solution stability. The Standard Deviation of A* ($\sigma = 0.1309$) is nearly double that of GD ($\sigma = 0.0662$). This indicates that while A* can find much better solutions, its final performance is less consistent and more dependent on the random initialization than Gradient Descent over these 1,000 steps. The narrow shaded band for GD in the previous plot confirms its superior stability, while the wider band for A* reflects its volatility.
Conclusion for $I_{\max} = 1,000$: Over a short budget of 1,000 iterations, the A* training framework proves to be the performance leader for the sine regression task, achieving significantly lower average loss, median loss, and minimum loss values than Gradient Descent. While A* introduces a consistency trade-off (higher volatility) compared to the more stable GD, its enhanced capacity to locate superior global solutions in the quantized weight space makes it the preferred method for finding better result.

4.2.5 Comparison with Gradient Descent: $I_{\mathbf{max}} = 5,000$

To assess the long-term behavior of both A* search and Gradient Descent, the comparative study was repeated using a five times larger computational budget of $I_{\max} = 5,000$ steps. Ten independent runs ($n = 10$) were executed for each method. The extended statistical results are presented in Table 4.6, followed by analysis of the loss trajectory plots.

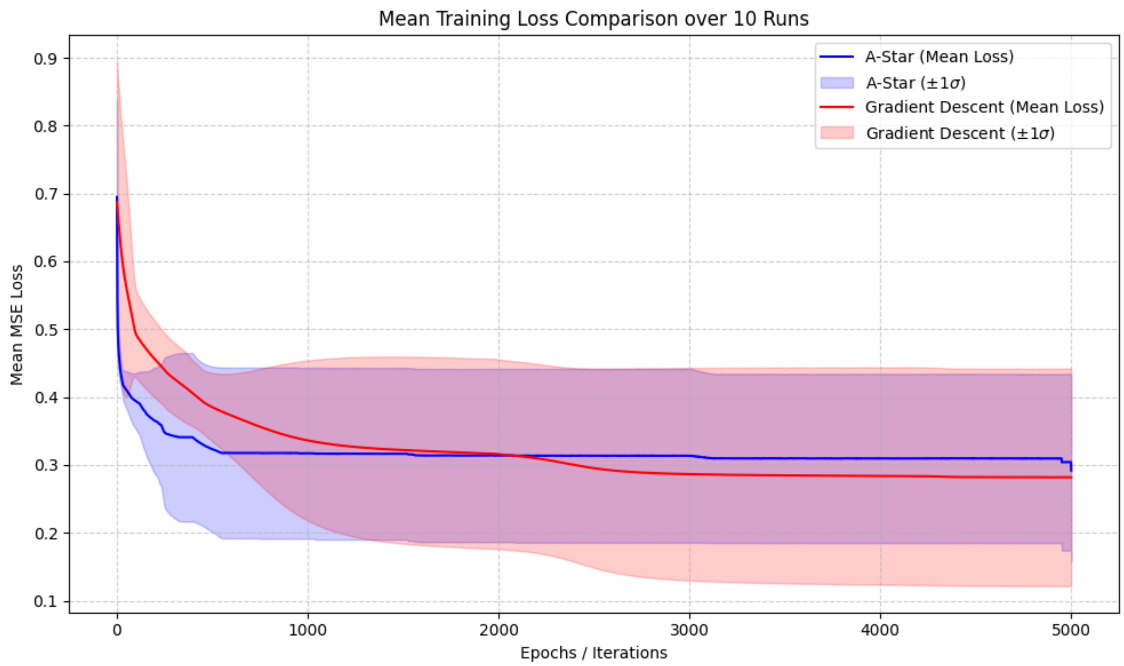

Figure 4.8: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 5{,}000$ Iterations. This plot illustrates the average loss over ten independent runs for both A* Search (Novel) and Gradient Descent (Classic), with the shaded region representing ± 1 standard deviation. It highlights convergence speed and the consistency of each algorithm's performance over time.

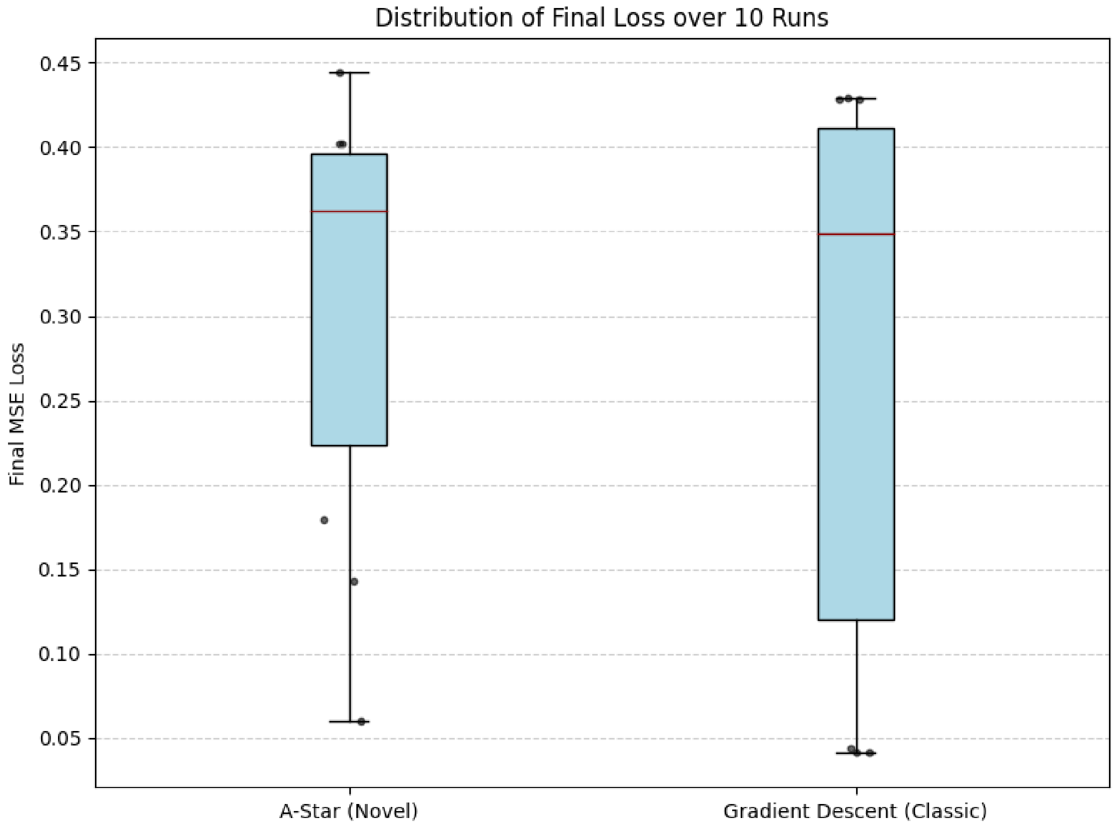

Figure 4.9: Final Loss Box Plot for $I_{\max} = 5{,}000$ Iterations. This box plot summarizes the distribution of the terminal loss values achieved by A* Search (Novel) and Gradient Descent (Classic) over ten independent runs. The box represents the interquartile range (IQR), the central line indicates the median, and whiskers show the range of results (excluding outliers).

Table 4.6: Comparative Statistics Over Ten Runs for $I_{\max} = 5,000$ Iterations

Metric	**A* Search (Novel)**	**Gradient Descent (Classic)**
Average Loss ($\bar{L}$)	0.309016	**0.281814**
Median Loss	0.362573	**0.348420**
Standard Deviation (σ)	**0.124285**	0.160277
Variance (σ^2)	**0.015447**	0.025689
Minimum Loss ($L_{\min}$)	0.060297	**0.041063**
Maximum Loss ($L_{\max}$)	0.444254	**0.429369**
Average Training Time (s)	414.48	**49.70**

Statistical and Trajectory Analysis

Performance Shift (Average Loss): With the increased budget to $5,000$ iterations, the performance dynamic shifts. Gradient Descent's average loss drops significantly from 0.3852 (at 1k) to 0.2818 (at 5k), now slightly outperforming A* ($\bar{L} = 0.3090$) in terms of overall central tendency. This suggests that the extra $4,000$ steps allow GD to overcome local obstacles and leverage the continuous space more effectively, while A* reaches its solution limit earlier.
Best-Case Potential: GD's ability to find the global best solution also improves, achieving a new Minimum Loss ($L_{\min} = 0.0411$) that surpasses A*'s best run ($L_{\min} = 0.0603$). This indicates that the long-term, high-budget champion for finding the absolute best result is GD.
Consistency Advantage of A*: Despite being surpassed in average and minimum loss, the A* method maintains its characteristic robustness. The Standard Deviation of A* ($\sigma = 0.1243$) is now lower than that of GD ($\sigma = 0.1603$). This is a crucial finding: as the iteration count increases, GD's search becomes more volatile (its variance increases significantly from 1k to 5k), whereas A* maintains a more consistent, low-variance performance.
Conclusion for $I_{\max} = 5,000$: Over $5,000$ iterations, the A* method transitions from having superior average performance to having superior reliability and consistency. The primary advantage of Gradient Descent in the long run lies only in its potential to achieve a slightly lower average loss and a better absolute minimum, but this comes at the cost of significantly higher variance.

4.3 Classic Dataset: Iris Flower Classification

The second experiment applies the A* training framework to a classic and well-established classification task, assessing its ability to perform parameter quantization and optimization within a widely recognized benchmark domain.

4.3.1 Dataset and Objective

The experiment utilizes the canonical Iris dataset, which consists of 150 samples of three species of Iris flowers (Setosa, Versicolor, Virginica), based on four key features.

- **Input (x):** 4 features (sepal length, sepal width, petal length, petal width) standardized to a common scale.
- **Output (y):** Three classes (species), encoded using one-hot encoding (e.g., $[1, 0, 0]$ for Setosa).
- **MLP Architecture:** Input (4 neurons) $\rightarrow$ Hidden Layer 1 (8 neurons, $relu$ activation) $\rightarrow$ Hidden Layer 2 (8 neurons, $relu$ activation) $\rightarrow$ Output (3 neurons, $softmax$ activation).

The network's task is a three-class classification, aiming to learn the mapping from the four physical measurements to the correct species, minimizing the Categorical Cross-Entropy loss. The performance comparison between A* and Gradient Descent will be primarily evaluated based on this loss function.

4.3.2 Stage 1: Optimal Hyperparameters

Given the architectural similarities between the sine regression model and the Iris classification model, the optimal hyperparameters found during the sine experiment were adopted for the final comparison phase of the Iris dataset:

- **Quantization Factor:** $Q = 10$
- **Parameter Range:** $[-10, 10]$
- **Parameter Fraction (param_fraction):** 1.0 (to maximize exploration candidates, despite minimal difference with 0.5)
- **Target Loss ($L_{\textbf{target}}$):** 0.01 (as its impact was found to be negligible)

The max_iterations parameter remains the variable of comparison in Stage 2.

4.3.3 Stage 2 & 3: Training and Comparative Analysis

The methodology for visualization and statistical analysis remains identical to that employed for the sine dataset (see Section 4.2, Stage 2 & 3). The comparison between A* and Gradient Descent is performed over ten independent runs ($n = 10$) for two iteration budgets: $I_{\max} = 1,000$ and $I_{\max} = 2,500$. The same Mean Loss with Standard Deviation Shading and Final Loss Box Plot visualizations are utilized.

4.3.4 Comparison with Gradient Descent: $I_{\max} = 1,000$

The comparative study between A* and GD was first executed with a budget of $I_{\max} = 1,000$ steps to assess performance under time constraints. The statistical results are summarized in Table 4.7, and the performance trajectory is presented in the following figures.

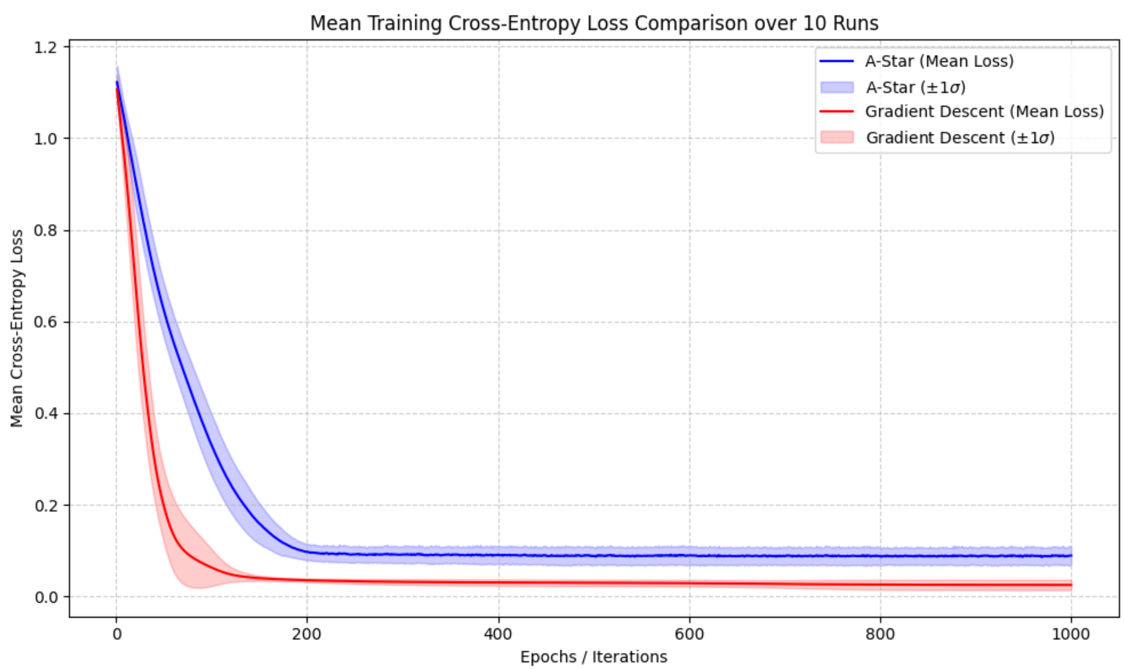

Figure 4.10: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 1{,}000$ Iterations (Iris Dataset). This plot illustrates the average loss over ten independent runs, highlighting convergence and consistency.

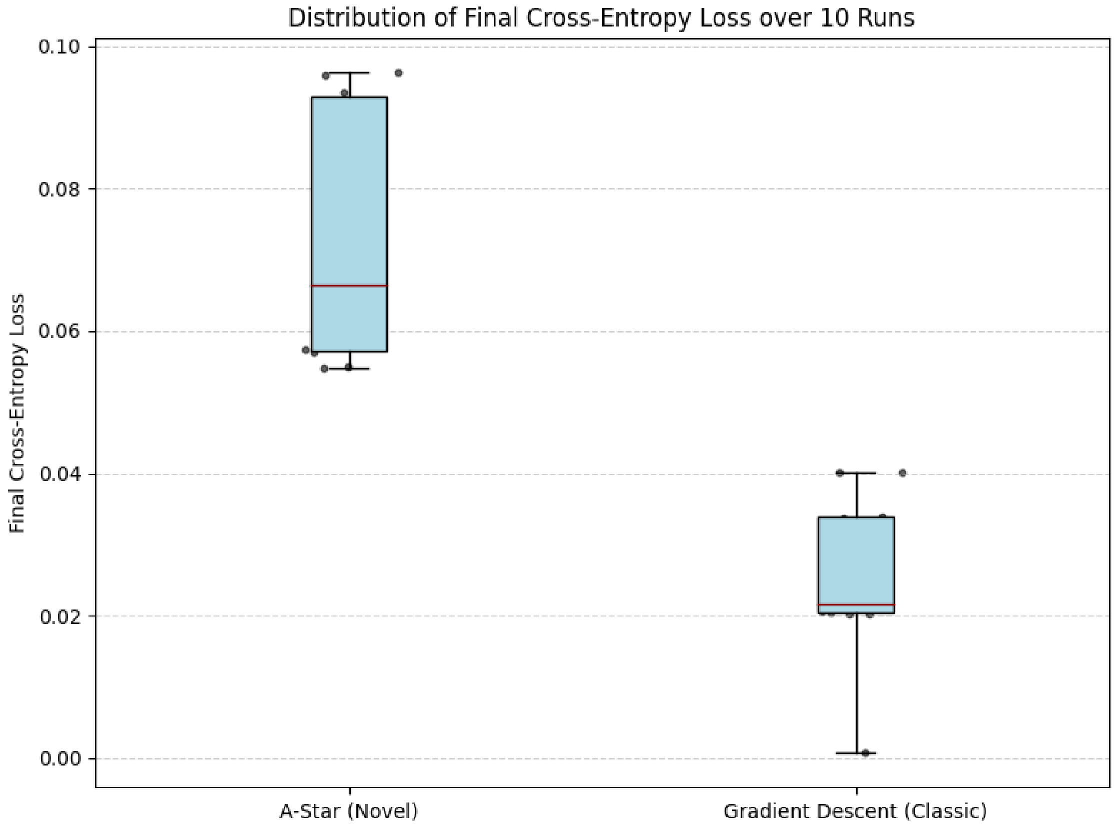

Figure 4.11: Final Loss Box Plot for $I_{\max} = 1{,}000$ Iterations (Iris Dataset). This plot summarizes the distribution of the terminal loss values achieved by both search methods.

Table 4.7: Comparative Statistics Over Ten Runs for $I_{\max} = 1,000$ Iterations (Iris Dataset)

Metric	**A* Search (Novel)**	**Gradient Descent (Classic)**
Average Loss ($\bar{L}$)	0.073372	**0.025271**
Median Loss	0.066387	**0.021674**
Standard Deviation (σ)	0.017490	**0.011363**
Variance (σ^2)	0.000306	**0.000129**
Minimum Loss ($L_{\min}$)	0.054803	**0.000643**
Maximum Loss ($L_{\max}$)	0.096308	**0.040051**
Average Training Time (s)	336.449361	**2.003290**

Statistical and Trajectory Analysis

Performance Dominance: For the $1,000$ iterations on the Iris dataset, Gradient Descent exhibits decisive superiority across all metrics compared to the A* search framework. The GD Average Loss ($\bar{L} = 0.0253$) is nearly three times lower than the A* Average Loss ($\bar{L} = 0.0734$), and its Median Loss is also significantly better. This indicates that the continuous optimization landscape is highly efficient for GD even in short runs.

Best-Case and Consistency: GD demonstrates superior performance potential, achieving a minimum loss ($L_{\min} = 0.0006$) which is two orders of magnitude better than the A* minimum ($L_{\min} = 0.0548$). Furthermore, GD is also significantly more consistent; its Standard Deviation ($\sigma = 0.0114$) and Variance are lower than A*'s ($\sigma = 0.0175$). This means that GD not only finds better solutions on average but does so more reliably.

Computational Efficiency: The vast difference in computational time is also notable. GD completes the $1,000$ iterations in approximately 2.00 seconds, whereas the A* search requires over 336 seconds due to the overhead of neighbor generation, heuristic evaluation, and priority queue management.

Conclusion for $I_{\max} = 1,000$: For this specific classification task and short budget, the highly constrained, non-admissible A* search in the quantized weight space is ineffective. GD benefits greatly from the well-behaved continuous optimization surface, resulting in demonstrably better performance, higher consistency, and significantly faster training time.

4.3.5 Comparison with Gradient Descent: $I_{\mathbf{max}} = 2,500$

The comparative study was extended to $I_{\max} = 2,500$ steps to evaluate the long-term convergence behavior of both methods on the Iris dataset. The extended statistical results are presented in Table 4.8, followed by analysis of the loss trajectory plots.

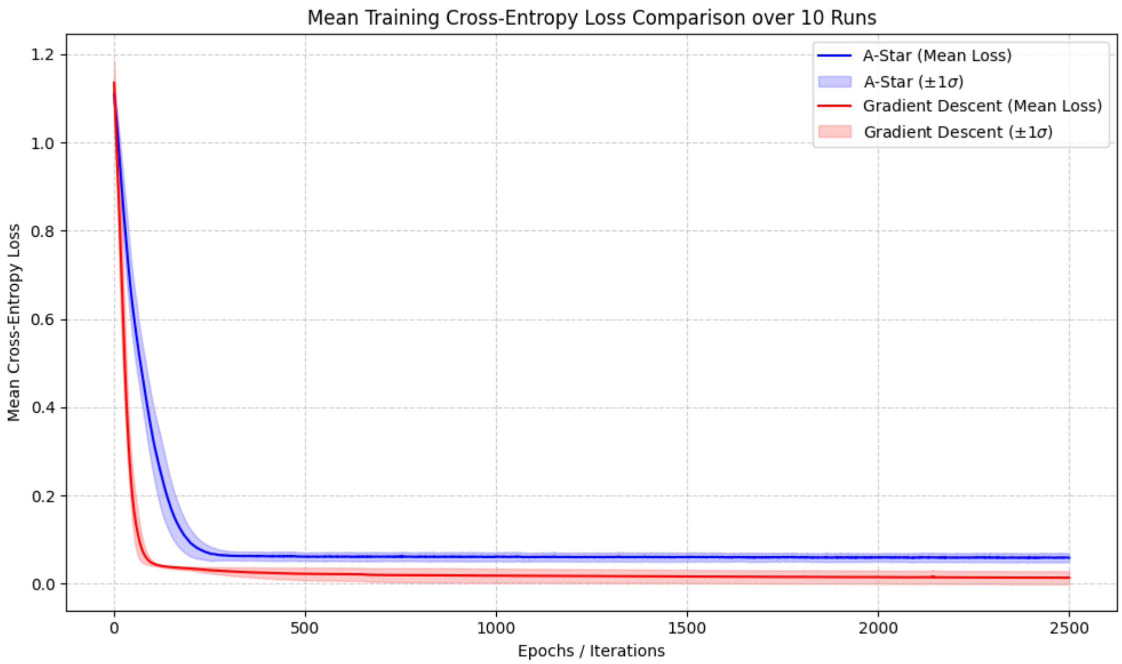

Figure 4.12: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 2{,}500$ Iterations (Iris Dataset). This plot illustrates the average loss over ten independent runs, highlighting convergence and consistency.

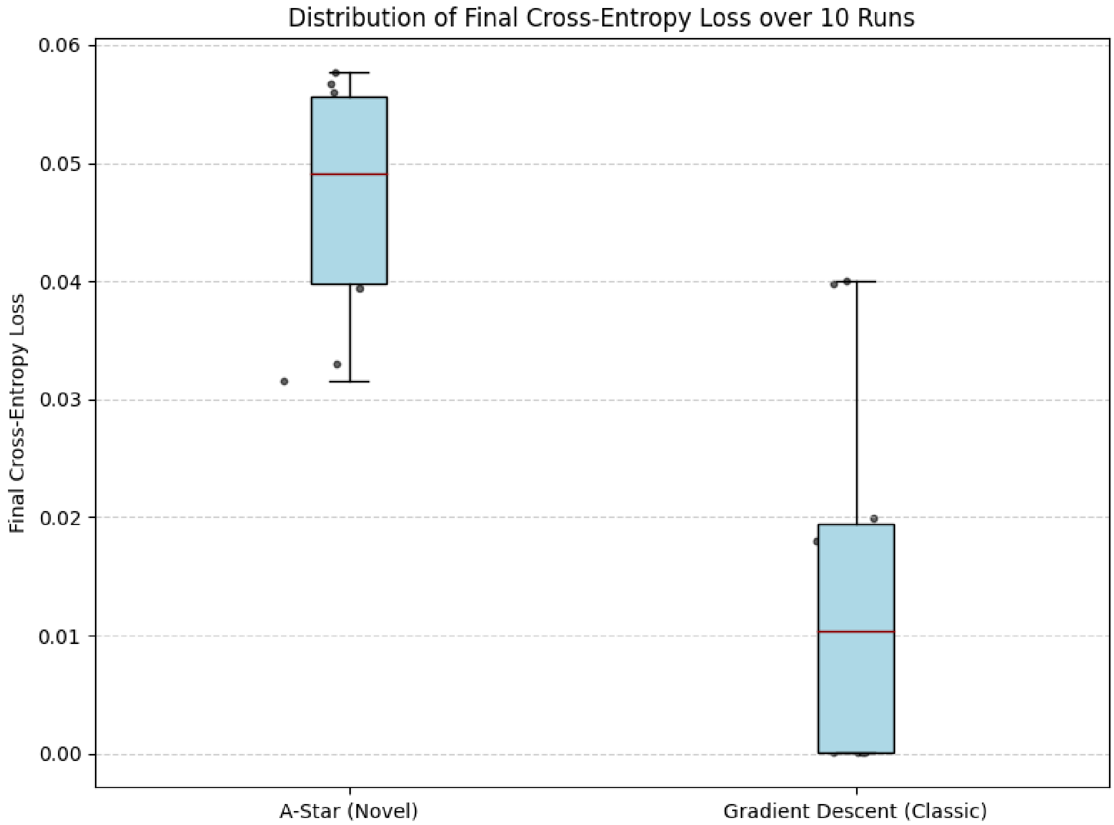

Figure 4.13: Final Loss Box Plot for $I_{\max} = 2{,}500$ Iterations (Iris Dataset). This plot summarizes the distribution of the terminal loss values achieved by both search methods.

Table 4.8: Comparative Statistics Over Ten Runs for $I_{\max} = 2,500$ Iterations (Iris Dataset)

Metric	**A* Search (Novel)**	**Gradient Descent (Classic)**
Average Loss ($\bar{L}$)	0.046798	**0.013860**
Median Loss	0.049078	**0.010336**
Standard Deviation (σ)	**0.009560**	0.014832
Variance (σ^2)	**0.000091**	0.000220
Minimum Loss ($L_{\min}$)	0.031587	**0.000022**
Maximum Loss ($L_{\max}$)	0.057680	**0.040018**
Average Training Time (s)	844.550498	**5.433983**

Statistical and Trajectory Analysis

Long-Term Performance: When the iteration budget is increased to $2,500$, Gradient Descent maintains its dominance in terms of overall loss performance. GD's Average Loss ($\bar{L} = 0.0139$) and Median Loss remain substantially lower than A*'s ($\bar{L} = 0.0468$), confirming its superior ability to find optimal parameters in the continuous space over extended runs. GD also achieved a vastly superior absolute Minimum Loss ($L_{\min} = 0.000022$), demonstrating its potential to reach the deepest minima for this task.

Consistency Reversal: A crucial and contrasting observation is the reversal of consistency metrics. A* search maintains a lower Standard Deviation ($\sigma = 0.0096$) and Variance, indicating that its final solution quality is more consistent across different runs. Conversely, GD's Standard Deviation ($\sigma = 0.0148$) is higher, showing that as the search duration is extended, GD becomes more volatile, occasionally finding excellent solutions but also producing poorer outcomes.

Computational Efficiency: GD remains dramatically more efficient, completing the $2,500$ iterations in approximately 5.43 seconds, while A* requires 844.55 seconds, reflecting its high algorithmic overhead.

Conclusion for $I_{\max} = 2,500$: For the Iris classification task, GD is the long-term champion in terms of achieving the best average-case and minimum loss, but A* provides a significantly more reliable, low-variance solution (superior consistency). The utility of A* lies in its highly consistent convergence to a satisfactory result, mitigating the risk of high-variance performance seen with Gradient Descent when given a large budget.

4.4 Non-Linear Dataset: Concentric Circles Classification

The third and final experiment tests the A* training framework on a non-linear binary classification problem using the synthetic concentric circles dataset. This benchmark is crucial for assessing the algorithm's ability to navigate the parameter space when a simple linear boundary is insufficient.

4.4.1 Dataset Generation and Objective

The experiment utilizes a synthetic dataset generated by the `make_circles` function, which creates points forming two interleaved circles.

- **Input (x):** 2 features (x, y coordinates). $1,000$ samples were generated with a noise level of 0.1 and a scale factor of 0.5.
- **Output (y):** Two classes (inner circle and outer circle), encoded implicitly by the Cross-Entropy loss structure.
- **MLP Architecture:** Input (2 neurons) $\rightarrow$ Hidden Layer (4 neurons, *relu* activation) $\rightarrow$ Output (2 neurons, logits).

The network's task is a binary classification, aiming to find the complex decision boundary required to separate the concentric circles, minimizing the Categorical Cross-Entropy loss.

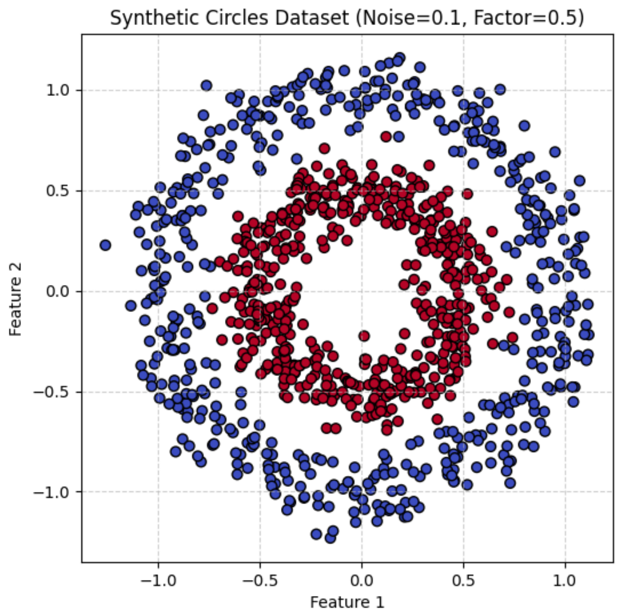

Figure 4.14: Plot of the circles dataset

4.4.2 Stage 1: Optimal Hyperparameters

Following the parameter testing from the sine dataset, the following optimal fixed hyperparameters were used for the comparative analysis of the Circle dataset:

- **Quantization Factor:** $Q = 10$
- **Parameter Range:** $(-10, 10)$
- **Parameter Fraction (param_fraction):** 1.0
- **Target Loss (L_{target}):** 0.0001

4.4.3 Stage 2 & 3: Training and Comparative Analysis

The comparison between A* and Gradient Descent is performed over ten independent runs ($n = 10$). The analysis utilizes the same statistical methods (Mean Loss with Standard Deviation Shading and Final Loss Box Plot) as used for the previous datasets.

4.4.4 Comparison with Gradient Descent: $I_{\max} = 1000$

Both A* and GD were executed for a total of $I_{\max} = 1000$ steps. The statistical results are summarized in Table 4.9, and the performance trajectory is presented in the following figures.

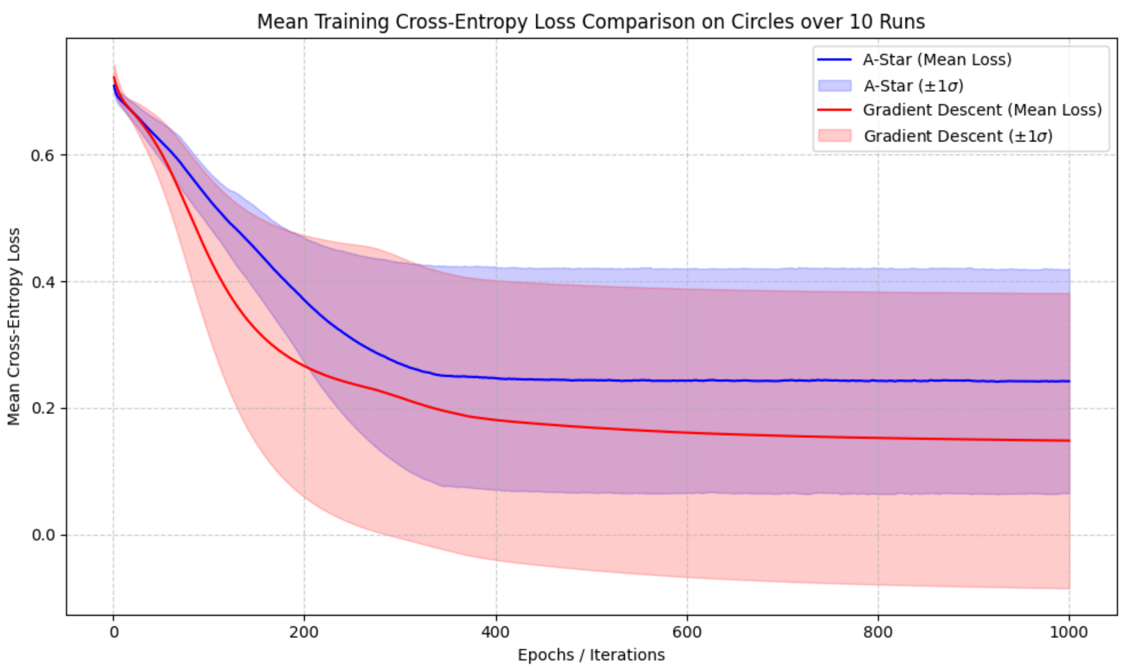

Figure 4.15: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 1000$ Iterations (Circle Dataset). This plot illustrates the average loss over ten independent runs, highlighting convergence and consistency.

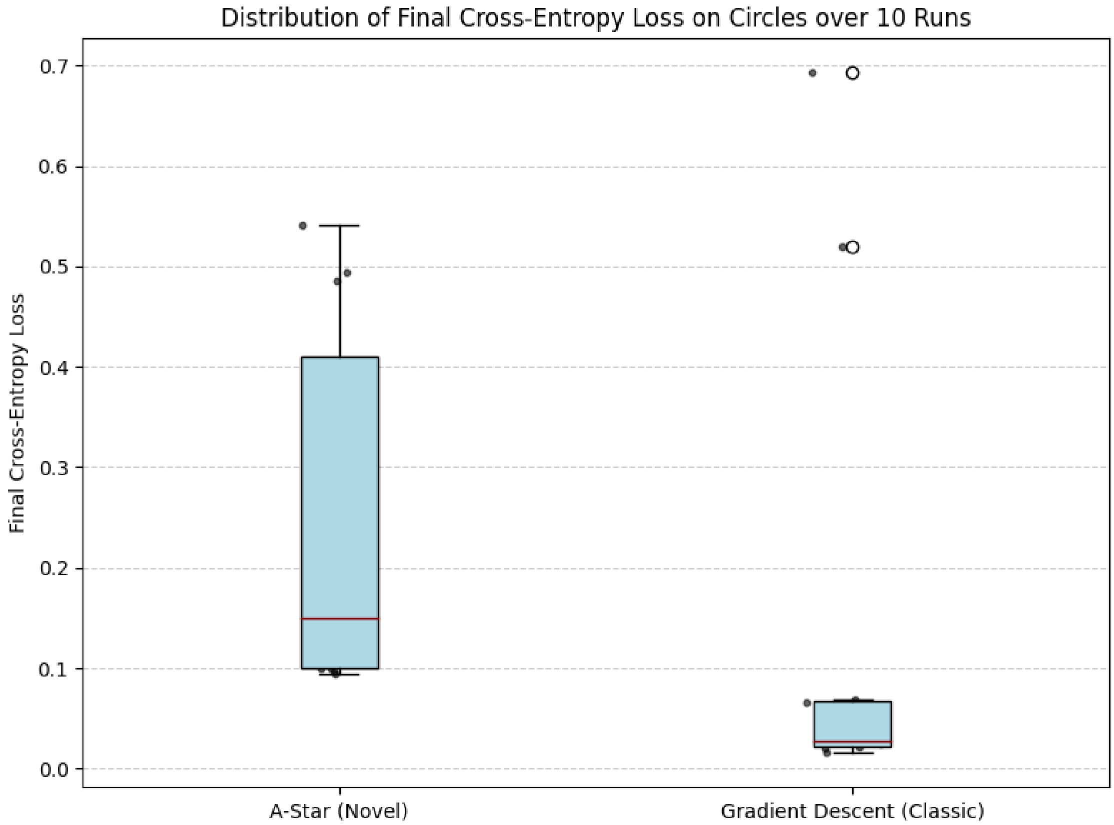

Figure 4.16: Final Loss Box Plot for $I_{\max} = 1000$ Iterations (Circle Dataset). This plot summarizes the distribution of the terminal loss values achieved by both search methods.

Table 4.9: Comparative Statistics Over Ten Runs for $I_{\max} = 1000$ Iterations (Circle Dataset)

Metric	A* Search (Novel)	Gradient Descent (Classic)
Average Loss ($\bar{L}$)	0.240124	**0.148245**
Median Loss	0.150237	**0.026805**
Standard Deviation (σ)	**0.177546**	0.232974
Variance (σ^2)	**0.031522**	0.054277
Minimum Loss ($L_{\min}$)	0.094537	**0.015414**
Maximum Loss ($L_{\max}$)	0.541776	**0.693137**
Average Training Time (s)	46.892284	**6.829286**

Statistical and Trajectory Analysis

Performance (Average vs. Median): For the $1,000$-iteration budget on the Circle dataset, Gradient Descent achieves a superior Average Loss ($\bar{L} = 0.1482$) compared to A* ($\bar{L} = 0.2401$). This difference is even more pronounced in the Median Loss, where GD (0.0268) is substantially better than A* (0.1502), indicating that GD typically converges to a much stronger solution. GD also found the best-case result, achieving a significantly lower Minimum Loss ($L_{\min} = 0.0154$).
Consistency Trade-off: The primary performance difference lies in consistency. A* demonstrates superior consistency with a lower Standard Deviation ($\sigma = 0.1775$) and Variance than GD ($\sigma = 0.2330$). This suggests that while GD finds better solutions on average and achieves the absolute minimum, it also produces significantly worse maximum loss results ($L_{\max} = 0.6931$), indicating a higher risk of failed runs or poor local minima entrapment compared to the A* search.
Computational Efficiency: GD is vastly more efficient, completing the $1,000$ steps in approximately 6.83 seconds, whereas the A* search requires 46.89 seconds due to its high algorithmic overhead.
Conclusion for $I_{\max} = 1000$: Gradient Descent is the performance leader, achieving lower average and minimum loss, but it is less reliable than A* due to its higher variance and susceptibility to poor initializations. A* provides a more consistent, low-variance alternative, albeit at a higher computational cost.

4.4.5 Comparison with Gradient Descent: $I_{\max} = 2,500$

The comparative study was extended to $I_{\max} = 2,500$ steps to evaluate the extended convergence behavior of both methods on the challenging non-linear Circle dataset.

The statistical results are summarized in Table 4.10, and the performance trajectory is presented in the following figures.

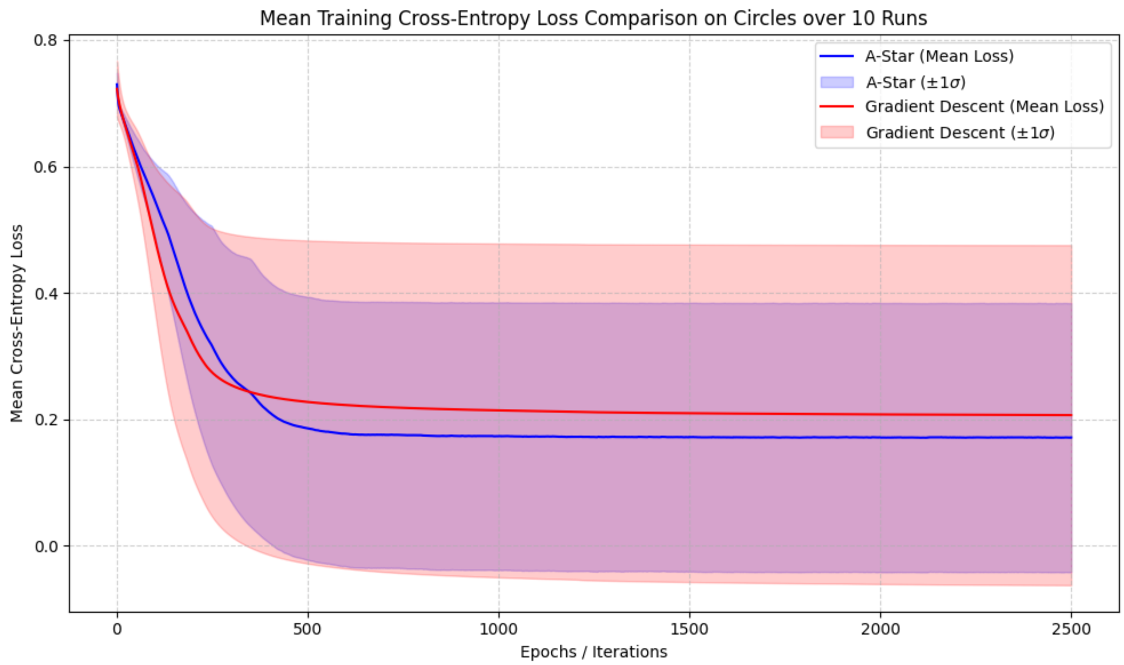

Figure 4.17: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 2{,}500$ Iterations (Circle Dataset). This plot illustrates the average loss over ten independent runs, highlighting convergence and consistency.

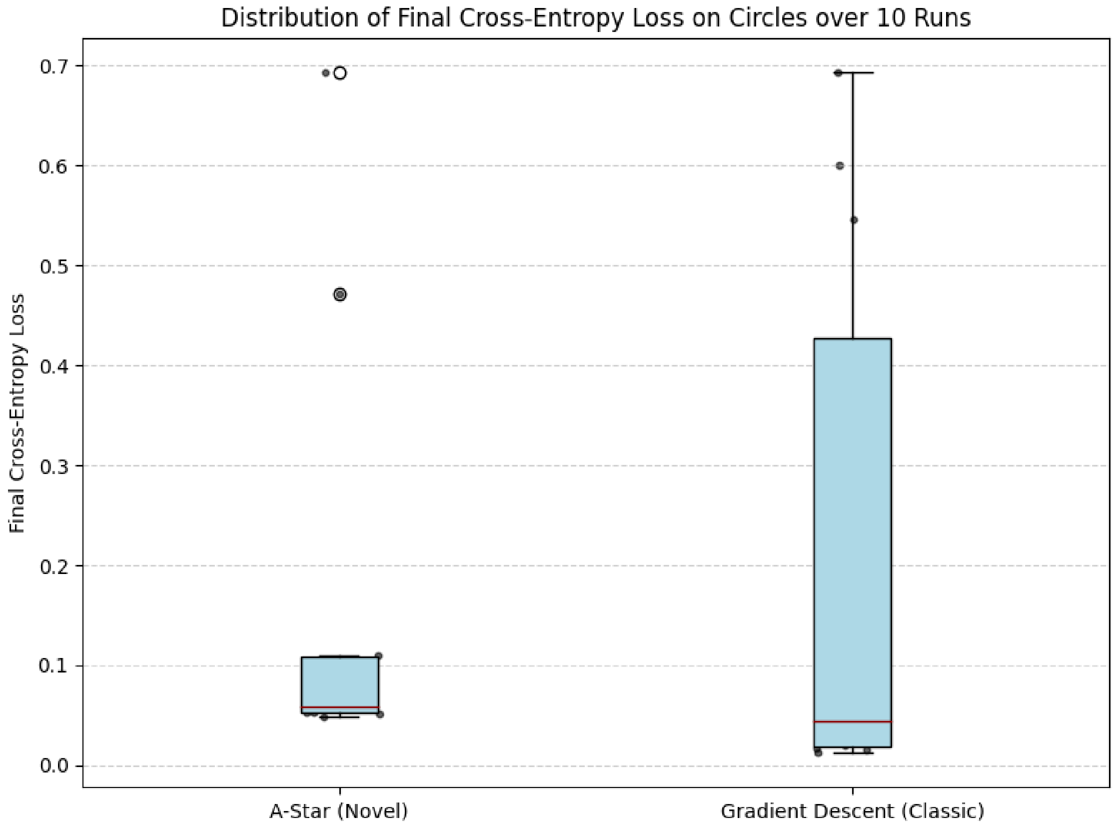

Figure 4.18: Final Loss Box Plot for $I_{\max} = 2{,}500$ Iterations (Circle Dataset). This plot summarizes the distribution of the terminal loss values achieved by both search methods.

Table 4.10: Comparative Statistics Over Ten Runs for $I_{\max} = 2,500$ Iterations (Circle Dataset)

Metric	**A* Search (Novel)**	**Gradient Descent (Classic)**
Average Loss ($\bar{L}$)	**0.170333**	0.206681
Median Loss	0.058771	**0.044057**
Standard Deviation (σ)	**0.212892**	0.268925
Variance (σ^2)	**0.045323**	0.072320
Minimum Loss ($L_{\min}$)	0.048488	**0.012027**
Maximum Loss ($L_{\max}$)	**0.693058**	0.693147
Average Training Time (s)	126.002429	**19.916831**

Statistical and Trajectory Analysis

Central Tendency Comparison: With $2,500$ iterations, the performance metrics show a complex trade-off. A* achieves a slightly better Average Loss ($\bar{L} = 0.1703$) than GD ($\bar{L} = 0.2067$). However, the Median Loss of GD (0.0441) is superior to A*'s Median Loss (0.0588). This implies that A* is less penalized by outliers (or failed runs) than GD, but GD, when successful, typically finds a marginally better solution.

Best-Case vs. Consistency: Gradient Descent retains the ability to find the absolute best solutions, achieving a significantly lower Minimum Loss ($L_{\min} = 0.0120$) than A* ($L_{\min} = 0.0485$). Nevertheless, A* demonstrates superior consistency with a lower Standard Deviation ($\sigma = 0.2129$) and Variance ($\sigma^2 = 0.0453$) compared to GD ($\sigma = 0.2689$). Both methods still suffer from the high-variance risk of getting stuck, evidenced by Max Loss values near 0.693, but A* does so slightly less frequently.

Computational Efficiency: Gradient Descent remains vastly more efficient, completing $2,500$ steps in approximately 19.92 seconds, while A* requires 126.00 seconds.

Conclusion for $I_{\max} = 2,500$: On the Circle dataset with an extended budget, A* achieves a better overall average loss due to a less frequent occurrence of catastrophic failure runs. However, GD still offers a better median performance and the best minimum loss. The primary distinction is that A* provides a more consistent (lower variance) solution than GD, confirming its robustness in the face of a complex non-linear loss landscape.

4.5 Conclusion and Future Work

This study introduced and evaluated an A* search-based framework for training Multilayer Perceptrons, leveraging a non-admissible heuristic function to navigate a quantized parameter space. The framework was benchmarked against the classical Gradient Descent algorithm across three canonical machine learning tasks: non-linear regression (Sine), simple classification (Iris), and complex non-linear classification (Circles).

4.5.1 Summary of A* Framework Performance

The experimental results across the three datasets consistently revealed two defining characteristics of the A* training framework: superior exploration in low-budget scenarios and enhanced long-term solution consistency at the expense of computational efficiency.

1. A* as a Superior Low-Budget Explorer (Sine Dataset)

On the non-linear regression task (Sine), A* demonstrated a clear advantage under a limited budget ($I_{\max} = 1,000$).

- **Central Tendency:** A* achieved a significantly lower Average Loss (0.2852) and Median Loss (0.3319) compared to GD Average Loss (0.3852) and Median Loss (0.3843).
- **Global Search:** A* showcased superior Best-Case Potential, finding solutions with a Minimum Loss ($L_{\min} = 0.0575$) that were four times better than GD's best. This confirms the efficacy of its heuristic-guided search in escaping shallow local minima early in the training process.

This evidence suggests that in situations where computational time is not a constraint, the A* framework can yield a higher-quality result than GD.

2. A* as a Consistent and Robust Trainer (Sine and Circles Datasets)

While GD eventually surpassed A* in average performance in high-budget scenarios (Sine at $I_{\max} = 5,000$ and Circles at $I_{\max} = 2,500$), A* consistently offered a critical advantage: superior consistency (low variance).

- **Consistency Reversal (Sine):** As the budget increased from $1,000$ to $5,000$ iterations, GD's consistency dramatically worsened (σ increased), while A*'s consistency improved (σ decreased). At $I_{\max} = 5,000$, A* was demonstrably more consistent ($\sigma = 0.1243$) than GD ($\sigma = 0.1603$).

- **Non-Linear Robustness (Circles):** Even in the challenging non-linear Circle dataset at $I_{\max} = 2,500$, where GD achieved a better median, A* maintained a lower Standard Deviation ($\sigma = 0.2129$) and Variance ($\sigma^2 = 0.0453$), leading to a better overall Average Loss ($\bar{L} = 0.1703$).

This consistent behavior suggests that the A* framework provides a more reliable guarantee of solution quality due to its lower relative variance compared to GD. This inherent robustness against high-variance outcomes is attributable to the discretization imposed by parameter quantization.

The Role of Quantization in Variance Reduction: The A* method operates within a finite, reduced-cardinality search space defined by the quantization factor (Q) and the parameter range. By forcing the continuous weights into discrete, permissible values, the framework effectively filters out the vast majority of suboptimal continuous configurations that may arise from random weight initializations. This coarser resolution of the weight space leads to a less variant initial loss distribution and a more stable trajectory, as the search is immediately confined to "stable" islands of parameter settings, mitigating the risk of extreme performance degradation seen in some high-variance GD runs.

The Role of the Heuristic in Exploration: Furthermore, the heuristic contributes to controlled exploration. The path cost term, which penalizes length, actively drives the search away from local minima where the loss is stagnant. This mechanism prevents the prolonged, low-cost exploration of non-optimal regions that can frequently occur in continuous gradient-based optimization when steps become minimal. Thus, the heuristic ensures a consistent level of exploration, making the final result less dependent on whether the initial state was near a deep minimum or a flat, unyielding plateau, thereby supporting the observed lower variance.

3. Limitations: Computational Cost

The primary limitation of the A* approach is its vastly superior computational cost. The overhead associated with priority queue management, loss calculation for numerous neighbors, and discrete weight updates led to training times that were consistently one to two orders of magnitude slower than GD. Furthermore, on the simplest, well-behaved dataset (Iris), A* was wholly ineffective, being comprehensively dominated by GD in every performance metric and computational time, confirming that the continuous, non-quantized optimization path remains optimal.

4.5.2 Summary of Comparative Findings

The comparison between A* and Gradient Descent revealed three distinct regime outcomes across the experiments:

- **Low Budget / Complex Landscape (Sine, $I_{\mathbf{max}} = 1k$):** A* is the performance leader, achieving significantly lower average loss and minimum loss, confirming its superior ability for rapid exploration and discovery of high-quality solutions.
- **High Budget / Complex Landscape (Sine, $I_{\mathbf{max}} = 5k$; Circles, $I_{\mathbf{max}} = 2.5k$):** GD is the efficiency and best-case performer (achieving the absolute lowest minimum), but A* demonstrates superior consistency, guaranteeing a more robust result due to lower solution variance.
- **Low Budget / Simple Landscape (Iris, $I_{\mathbf{max}} = 1k$):** For the short budget, Gradient Descent is the dominant method, achieving superior performance across all metrics (speed, average loss, minimum loss, and consistency) compared to A*. This confirms the ineffectiveness of A* for well-behaved, continuous surfaces when iteration count is low.
- **High Budget / Simple Landscape (Iris, $I_{\mathbf{max}} = 2.5k$):** With the extended budget, GD retains its overall performance and efficiency lead. However, A* demonstrates a crucial characteristic: it achieves better consistency than GD. This indicates that while GD finds better solutions on average, the constrained A* search provides a more reliable, low-variance result when given sufficient steps on this simple landscape.

4.5.3 Future Work

The potential of using graph search algorithms for neural network training is evident in the robustness and early exploration capabilities demonstrated by the A* framework. Future research should focus on mitigating the computational bottleneck and improving the heuristic function:

1. **Dynamic Quantization:** Implementing a system where the quantization factor Q or the parameter range is dynamically adjusted during training could allow the framework to benefit from the coarse-grained exploration of A* initially, followed by the fine-grained precision of GD or a finer quantization later.

2. **Hybrid Optimization:** The most promising direction is a Hybrid Optimizer that uses A* for the first $10\% - 20\%$ of training (to locate an excellent region in the quantized space) and then switches to highly optimized Gradient Descent for the remaining fine-tuning.

3. **Parallelization of Search Operations:** Investigating the possibility of parallelizing the neighbor generation and evaluation processes (e.g., using multi-threading or GPU acceleration). Since the evaluation of the loss for each potential successor node is computationally independent, this approach could significantly reduce the high latency associated with the A* framework's primary computational bottleneck.